

1. Características de GNU/Linux:

a. Mencione y explique las características más relevantes de GNU/Linux.

b. Mencione otros sistemas operativos y compárelos con GNU/Linux en cuanto a los puntos mencionados en el inciso a.

c. ¿Qué es GNU?

d. Indique una breve historia sobre la evolución del proyecto GNU.

e. Explique qué es la multitarea, e indique si GNU/Linux hace uso de ella.

f. ¿Qué es POSIX?

a.

- Es un sistema operativo tipo Unix (Unix-like) pero libre. Fue diseñado de manera colaborativa por miles de programadores alrededor del mundo.
- **Es Software Libre:** Respeta libertades fundamentales del usuario: se puede usar, copiar, estudiar, modificar y redistribuir libremente.
- **Disponibilidad del código fuente:** Se distribuye de forma abierta y comúnmente junto con su código fuente. Esto permite estudiarlo, personalizarlo, auditarlo y posibilita una corrección de fallas mucho más rápida al estar expuesto a la revisión comunitaria.
- **Organización en Distribuciones:** El sistema se organiza y ofrece a través de diversas "distribuciones" (customizaciones), compuestas por una versión específica del núcleo (Kernel), bibliotecas y determinados programas con sus respectivas configuraciones.
- **Es Multitarea, Multiusuario y Multiprocesador:** Permite que múltiples usuarios compartan recursos y que se ejecuten diversas tareas concurrentemente sobre uno o más procesadores.
- **"Todo es un archivo":** En GNU/Linux existe el concepto de que prácticamente todo elemento del sistema, inclusive los dispositivos de hardware y los directorios, se administra y representa bajo la estructura de un archivo.
- **Estructura de almacenamiento flexible:** Es altamente portable. Además, cada directorio del sistema de archivos puede estar montado en una partición de disco diferente (por ejemplo, /home o /tmp).
- **Sensibilidad a las mayúsculas (Case Sensitive):** El sistema operativo distingue estrictamente entre mayúsculas y minúsculas en nombres de archivos, directorios y comandos.
- **Posee intérpretes de comandos (Shell/CLI):** Cuenta con diversas interfaces de línea de comandos programables y personalizables (como Bash, Korn Shell, etc.) que actúan como modo de comunicación directo entre el usuario y el S.O

b.

- **Costo de adquisición:** El software propietario generalmente tiene un costo económico o de licencia asociado, mientras que el software libre suele ser de costo nulo (aunque la cátedra resalta que es un error asociar el software libre únicamente con gratuidad, ya que el foco está en las libertades otorgadas).
- **Distribución y copia:** El software propietario prohíbe su libre distribución y copia, mientras que GNU/Linux fomenta la redistribución de copias sin restricciones.
- **Modificación y Acceso al Código:** En los sistemas propietarios normalmente no se distribuye el código de origen y no se permiten modificaciones. GNU/Linux es de código abierto y provee el código fuente para que cualquiera pueda estudiarlo y mejorarlo.
- **Soporte y corrección de fallas:** En los S.O. propietarios, la detección y corrección de errores depende exclusivamente de la empresa dueña del software. En GNU/Linux, al contar con una comunidad masiva que accede al código fuente, los parches de seguridad y correcciones ante fallas se desarrollan con mayor velocidad.
- **Requerimientos técnicos:** El software propietario suele demandar menos técnicos especializados para su administración diaria, mientras que el software libre a menudo requiere perfiles técnicos más capacitados.

c.

GNU es un acrónimo recursivo que significa "GNU No es Unix".

Fue un proyecto iniciado por Richard Stallman en 1983 con el objetivo fundamental de crear un sistema operativo compatible con Unix que fuera completamente libre (el sistema GNU).

d.

- **1983:** Richard Stallman anuncia formalmente el proyecto para construir el sistema operativo libre GNU.
- **1985:** Se establece la Free Software Foundation (FSF) para dar soporte financiero al desarrollo.
- **1988:** Se intenta desarrollar un núcleo basado en un proyecto llamado TRIX, pero se abandona por su alta complejidad. En su lugar, se adopta el micronúcleo MACH para dar forma al proyecto de núcleo GNU Hurd, el cual tampoco prosperó.
- **1989:** La FSF publica la primera versión de la licencia GPL para blindar los desarrollos del proyecto.

- **1990:** El ecosistema de GNU está prácticamente completado en su espacio de usuario (compiladores, utilidades, editores), pero sigue sin disponer de un Kernel funcional.
- **1991:** De forma paralela e independiente, un estudiante llamado Linus Torvalds comienza a desarrollar un núcleo basado en el S.O. académico Minix, licenciándolo bajo la GPL.
- **1992 (Nacimiento de GNU/Linux):** Torvalds y Stallman acuerdan fusionar el Kernel Linux con el conjunto de utilidades y compiladores del sistema GNU. A partir de esta fusión nace el sistema operativo completo denominado académicamente GNU/Linux

e.

Sí, GNU/Linux es un sistema operativo multitarea

- **Concepto:** En sistemas primitivos por lotes (batch), la CPU quedaba ociosa esperando operaciones de entrada/salidas muy lentas. Con la llegada de la multiprogramación, el S.O. pasó a mantener múltiples procesos o tareas cargados en la memoria de manera simultánea. Cuando un proceso se detiene a realizar una operación de entrada/salida (E/S), la CPU no se queda esperando; el sistema operativo le cede el procesador a otro proceso que esté listo para ejecutarse.
- **Tiempo Compartido:** La multitarea moderna aplica esta lógica a través del tiempo compartido, repartiendo de forma equitativa el uso del procesador entre múltiples tareas interactivas. El sistema asigna a cada proceso la CPU durante un periodo máximo de tiempo (conocido como quantum o fracción de tiempo). Al vencer ese tiempo, el control se le retira a ese proceso y se le entrega a otro en la fila de espera. Al ser esta alternancia increíblemente veloz, el usuario percibe que todos los programas se ejecutan en forma simultánea (conurrencia).

f.

- POSIX es el acrónimo de Portable Operating System Interface (con la "X" agregada históricamente por su estrecha herencia de UNIX).
- Es una familia de estándares e interfaces formalizados por la IEEE para definir un estándar de compatibilidad y comunicación entre sistemas operativos Unix y similares (Unix-like).
- **Función:** Define una interfaz estandarizada para llamadas al sistema (APIs de programación en C), herramientas de comandos de consola, intérpretes de comandos (Shell), y gestión de procesos e hilos (pthreads).

- Importancia: Al programar un código bajo los lineamientos de POSIX, se asegura la portabilidad del software a nivel de código fuente. Esto garantiza que el software se pueda compilar y ejecutar con mínimos cambios en cualquier sistema que cumpla con el estándar (como Linux, macOS o BSD).

2. Distribuciones de GNU/Linux:

- a. ¿Qué es una distribución de GNU/Linux? Nombre al menos 4 distribuciones de GNU/Linux y cite diferencias básicas entre ellas.
- b. ¿En qué se diferencia una distribución de otra?
- c. ¿Qué es Debian? Acceda al sitio <https://www.debian.org/> e indique cuáles son los objetivos del proyecto y una breve cronología del mismo.

a.

Una distribución es una customización de GNU/Linux formada por una versión del núcleo (Kernel) y un determinado paquete de programas (utilidades) con sus respectivas configuraciones. Las utilidades incluidas son el componente que permite diferenciar una distribución de otra.

4 distribuciones y sus diferencias básicas:

1. **Debian:** Destacada por su extrema estabilidad, robustez y su política sumamente estricta de no incluir software propietario de forma predeterminada. Es una de las distribuciones más antiguas e independientes, mantenida por una comunidad global de voluntarios.
2. **Ubuntu:** Basada en Debian, pero enfocada en la facilidad de uso y la amigabilidad para el usuario final (tanto en escritorio como en servidores). A diferencia de Debian, incluye de forma más directa controladores propietarios (como los de placas de video Nvidia) y software comercial para facilitar la experiencia a principiantes.
3. **Fedora / Red Hat Enterprise Linux (RHEL):** Distribuciones fuertemente orientadas al entorno corporativo y empresarial (RHEL es el estándar comercial y Fedora es su banco de pruebas comunitario). Utilizan un gestor de paquetes diferente (RPM en lugar del DPKG de Debian) y se caracterizan por integrar tecnologías de vanguardia muy rápidamente.
4. **Arch Linux / Gentoo:** Distribuciones orientadas a usuarios avanzados. Arch Linux utiliza un modelo de actualización continua (*rolling release*, sin versiones mayores) donde el usuario instala todo a mano desde una consola. Gentoo va un paso más allá y requiere que el usuario compile

cada programa desde el código fuente en su propia máquina para optimizar el rendimiento al máximo.

b.

- **Las utilidades y paquetes de software:** Los entornos de escritorio por defecto (como GNOME, KDE, XFCE o MATE) y el conjunto de aplicaciones preinstaladas.
- **El sistema de gestión de paquetes:** La familia Debian/Ubuntu utiliza archivos .deb (gestionados mediante dpkg y apt), mientras que la familia Red Hat/Fedora utiliza archivos .rpm (gestionados por rpm y dnf).
- **La filosofía frente al software propietario:** Algunas distribuciones prohíben estrictamente incluir cualquier tipo de código cerrado en sus repositorios principales, mientras que otras priorizan la comodidad del usuario facilitando la instalación de códecs de música, controladores gráficos y programas propietarios.
- **El ciclo de vida y modelo de lanzamientos:** Existen distribuciones con lanzamientos programados fijos (por ejemplo, una versión nueva cada 2 años con soporte a largo plazo) y otras de actualización continua (*rolling release*) que nunca requieren reinstalar el sistema porque se actualizan día a día.

c.

Debian es una organización de voluntarios de alcance mundial dedicada a desarrollar un sistema operativo completamente libre. El nombre "Debian" fue creado por su fundador, Ian Murdock, combinando el nombre de su esposa en ese momento (Debra) con el suyo (Ian).

Objetivos del Proyecto Debian: Según su declaración de principios y su sitio web oficial, los objetivos principales son:

1. **Crear el "Sistema Operativo Universal":** Desarrollar un sistema de altísima calidad que sea capaz de correr en la mayor variedad de hardware posible (soportando arquitecturas de 32 y 64 bits como x86, ARM, PowerPC, MIPS) y admitiendo diferentes núcleos (como el Kernel de Linux o el de FreeBSD).

2. **Compromiso estricto con el Software Libre:** Todo el software del sistema principal debe cumplir estrictamente con el Contrato Social de Debian y las Directrices de Software Libre de Debian (DFSG), garantizando que el usuario siempre mantenga el control sobre su tecnología.
3. **Estabilidad y Confianza:** Mantener un proceso riguroso de control de calidad. Por ello, Debian divide su software en tres ramas activas: *unstable* (desarrollo rápido), *testing* (en pruebas) y *stable* (producción oficial). Solo el software altamente probado se publica en la versión *stable*.
4. **Gobierno Democrático:** El proyecto no pertenece a ninguna empresa comercial; se rige de manera independiente por una Constitución propia y un sistema de elecciones democráticas para elegir a sus líderes.

Breve Cronología del Proyecto:

5. **16 de Agosto de 1993:** Ian Murdock anuncia oficialmente la creación del proyecto Debian cuando el concepto de "distribución" de Linux era casi inexistente.
6. **1994 - 1995:** El proyecto es patrocinado por la Free Software Foundation (FSF) de Richard Stallman, lo que ayuda a asentar sus bases.
7. **Noviembre de 1995:** Se lanza Debian 0.93 Release 6, la última versión en formato antiguo *a.out*. En esta versión hace su aparición por primera vez la herramienta interactiva de paquetes *dselect*.
8. **1996:** Ian Murdock cede el liderazgo del proyecto. Se lanza la versión 1.1 para evitar confusiones de mercado (la versión 1.0 nunca se distribuyó oficialmente por el error de un fabricante de CDs).
9. **1997:** Se publica el Contrato Social de Debian, cuyas pautas sirvieron posteriormente como base para redactar la definición mundial de lo que hoy conocemos como *Open Source* (Código Abierto).
10. **2015:** Fallece Ian Murdock, el histórico fundador del proyecto.
11. **Actualidad:** El proyecto sigue sumamente activo, lanzando periódicamente sus versiones estables (nombradas siempre en honor a

los personajes de la película *Toy Story*). La versión estable actual es Debian 13 "Trixie", lanzada en agosto de 2025.

3. Estructura de GNU/Linux:

a. Nombre cuáles son los componentes fundamentales de GNU/Linux.

b. Mencione y explique la estructura básica del Sistema Operativo GNU/Linux.

a.

1. **Kernel (Núcleo):** Es el componente central del sistema operativo, el cual reside de forma permanente en la memoria principal. En un sentido estricto, es el sistema operativo en sí mismo, ya que se encarga de la interacción directa con el hardware y la administración de los recursos físicos de la computadora.
2. **Shell (Intérprete de comandos / Interfaz de usuario):** Es la interfaz que actúa como modo de comunicación directa entre el usuario y el sistema operativo. Su función principal es capturar los comandos ingresados por el usuario para mandar a ejecutar los programas. Puede presentarse en formato de consola de texto (CLI/CUI) o mediante entornos visuales de ventanas (GUI).
3. **Herramientas (Utilidades):** Representan el conjunto de programas necesarios para realizar tareas diarias y de desarrollo. Incluye los compiladores (como GCC), bibliotecas de sistema, utilidades de red, entornos de escritorio y editores de texto (como vi o emacs). Estas utilidades conforman el paquete de software que permite diferenciar a una distribución de otra

b.

La estructura de GNU/Linux está diseñada bajo un modelo de desarrollo en capas con una clara separación de funciones, donde cada nivel actúa como una "caja negra" hacia las demás capas. Esta estructura jerárquica organiza los recursos de la siguiente manera:

- **Capa Física (Hardware):** Situada en la base del sistema, se compone de los elementos físicos esenciales: el procesador (CPU), la memoria principal (RAM), los buses del sistema y los componentes de Entrada/Salida (placas de red, discos, monitores, periféricos).
- **Kernel (Modo Privilegiado / Modo Kernel):** Es la primera capa de software por encima del hardware. El núcleo se ejecuta con el bit de CPU en modo supervisor o kernel, lo que le otorga permisos absolutos para acceder directamente a la electrónica y ejecutar instrucciones privilegiadas. Sus funciones primordiales son la administración de la memoria (física y virtual), la planificación de la CPU, la administración de procesos, la comunicación y concurrencia entre ellos, y el control de los dispositivos de Entrada/Salida.

- **Llamadas al Sistema (System Calls) y Bibliotecas base:** Esta capa actúa como un puente intermedio de abstracción. Dado que las aplicaciones de usuario no pueden interactuar directamente con el hardware por cuestiones de seguridad, deben realizar solicitudes controladas al núcleo utilizando llamadas al sistema (System Calls), las cuales actúan como servicios expuestos por el S.O.
- **Espacio de Usuario (Modo Usuario):** Es la capa superior donde se ejecutan las utilidades, compiladores, editores de texto y todas las aplicaciones de usuario final. Esta capa opera en modo usuario, un subconjunto de instrucciones limitadas donde los procesos solo pueden interactuar con sus propios espacios de memoria asignados y no tienen permitido manipular directamente los dispositivos o el hardware general.
- **Shell (CLI / GUI):** Es la capa más externa del sistema operativo. Ya sea a través de la terminal ingresando comandos de texto (como sh, ksh o bash) o mediante gestores de interfaces gráficas (como GNOME o XFCE), es el entorno que permite a los seres humanos interactuar, lanzar procesos y controlar la computadora de manera cómoda y eficiente.

4. Kernel:

a. ¿Cuáles son sus funciones principales?

b. ¿Es posible tener más de un Kernel de GNU/Linux instalado en la misma máquina?
(ayuda /boot)

c. ¿En qué directorio se encuentra ubicado?

a.

El Kernel es el componente central del sistema operativo que se encuentra siempre residente en la memoria principal. En un sentido estricto, es el sistema operativo en sí mismo y su propósito principal es permitir que el hardware y el software trabajen juntos de manera coordinada.

Sus funciones principales se ejecutan en Modo Kernel (o supervisor) e incluyen:

1. **Administración de memoria:** Se encarga de gestionar de forma eficiente la memoria física (RAM) y la memoria virtual (la combinación de RAM y el espacio de intercambio o SWAP), reservando el espacio de direcciones para cada proceso y protegiéndolos de accesos no autorizados.
2. **Administración y planificación de la CPU:** Multiplexa la carga de trabajo y planifica qué procesos se ejecutarán en el procesador. Utiliza mecanismos de protección (como la interrupción por *clock*) para evitar que un solo proceso se apropie indefinidamente de la CPU.
3. **Gestión de Procesos:** Controla la creación, planificación, sincronización, comunicación y terminación de los procesos del sistema.
4. **Gestión de Entrada/Salida (E/S) y Dispositivos:** Administra los buffers, controla los canales de comunicación física y oculta la complejidad de los controladores de hardware (*drivers*), ofreciendo abstracciones más simples a las aplicaciones de usuario.
5. **Funciones de soporte interno:** Controla y gestiona el vector de interrupciones del hardware, la auditoría y el monitoreo del sistema.

b.

Sí, es totalmente posible y es una práctica estándar en GNU/Linux.

Las diferentes imágenes de los núcleos instalados se almacenan bajo nombres únicos (generalmente con el prefijo `vmlinuz-` seguido de su número de versión) dentro del directorio o partición `/boot`.

Durante el proceso de arranque (*bootstrap*), una vez que finaliza la ejecución del firmware (BIOS o UEFI), entra en acción el gestor de arranque (como GRUB). Este lee su archivo de configuración y presenta un menú interactivo en pantalla donde el usuario puede seleccionar explícitamente qué Kernel desea cargar en memoria para iniciar la sesión.

c.

Se encuentra ubicado físicamente en el directorio `/boot` (el cual suele configurarse de forma recomendada en su propia partición dedicada para mayor seguridad).

Dentro de este directorio vas a encontrar el archivo ejecutable del Kernel real, el cual habitualmente se identifica bajo la nomenclatura de archivo `vmlinuz-[versión_del_núcleo]` (por ejemplo: `vmlinuz-2.6.26`)

5. Intérprete de comandos (Shell):

a. ¿Qué es y cuáles son sus funciones?

b. Investigue y mencione al menos 3 intérpretes de comandos que existen para GNU/Linux y compárelos entre ellos.

c. ¿Dónde se ubican (path) los comandos propios y externos al Shell?

d. ¿Por qué considera que el Shell no es parte del Kernel de GNU/Linux?

e. ¿Es posible definir uno distinto para cada usuario?

a.

También conocido como CLI (Command Line Interface), es un componente de software que actúa como el modo de comunicación directa entre el usuario y el sistema operativo. Representa la capa más externa en la estructura del S.O.

Funciones principales:

- **Interacción del usuario:** Ofrece un entorno para que el usuario controle e interactúe con el sistema.
- **Ejecución de programas:** Su función primordial es capturar los comandos de texto ingresados por el usuario y mandar a ejecutar los programas correspondientes.
- **Automatización y personalización:** Permite personalizar el entorno de trabajo y, al ser programable, posibilita la creación de scripts (Shell Scripting) para automatizar tareas repetitivas o administrativas del sistema.}

b.

1. **Bourne Shell (sh):** Fue el primer shell estándar desarrollado para Unix. Es sumamente simple, rápido y altamente portable (es el estándar mínimo garantizado en casi cualquier Unix/Linux), pero carece de características interactivas modernas que resulten cómodas para el usuario (como historial de comandos accesible con las flechas o autocompletado avanzado).
2. **Korn Shell (ksh):** Creado como un superconjunto compatible con sh. Añade notables mejoras orientadas a la programación y utilidades interactivas que facilitan el trabajo del día a día, siendo muy utilizado en entornos Unix comerciales tradicionales.
3. **Bourne Again Shell (bash):** Es el intérprete de comandos por excelencia y predeterminado en casi todas las distribuciones modernas de GNU/Linux (como el Debian que instalaste). Es un desarrollo libre del proyecto GNU que destaca por incorporar funciones interactivas potentes como el autocompletado de

comandos y rutas (usando la tecla Tab), un sistema de historial de comandos (*history*) y el soporte para crear alias (atajos de comandos personalizados).

c.

Comandos propios (internos o *built-ins*): Son comandos que están programados directamente dentro del propio código ejecutable de la Shell. No existen como un archivo binario separado en el disco duro, por lo tanto, no tienen un path físico individual en el sistema de archivos; se cargan en la RAM junto con la Shell y se ejecutan de manera instantánea (ejemplos: `cd`, `pwd`, `exit`, `alias`).

Comandos externos: Son aplicaciones o programas independientes almacenados en forma de archivos ejecutables en el disco rígido. Su ubicación sigue el estándar FHS (*Filesystem Hierarchy Standard*). Los paths principales donde se ubican son:

- **/bin:** Archivos binarios y ejecutables esenciales indispensables para el funcionamiento básico del sistema y todos los usuarios (ejemplos: `ls`, `mkdir`, `cp`, `mv`).
- **/usr/bin:** Comandos y aplicaciones adicionales de usuario que no son críticos para el inicio básico del sistema (ejemplos: `vim`, `nano`, `python`).
- **/sbin y /usr/sbin:** Ejecutables reservados para tareas de administración del sistema y utilizados principalmente por el superusuario `root` (ejemplos: `useradd`, `fdisk`, `reboot`).

d.

Basado en los principios de diseño de sistemas operativos que detalla la cátedra, el Shell no forma parte del Kernel debido a tres razones fundamentales:

1. **Separación de funciones y diseño en capas:** El sistema se construye bajo un modelo de capas jerárquicas e independientes. El Kernel reside en la capa inferior gestionando el hardware de manera interna, mientras que el Shell pertenece a la capa más externa, sirviendo únicamente como puente de comunicación con el usuario.
2. **Diferencia de modos de ejecución:** El Kernel se ejecuta en Modo Supervisor (o Kernel), lo que les da acceso irrestricto a todas las instrucciones de la CPU y control físico sobre el hardware. El Shell se ejecuta en Modo Usuario. El Shell no puede manipular el hardware de forma directa; si necesita realizar una acción (como leer un archivo o escribir una línea en la pantalla), debe solicitarlo de forma controlada al núcleo mediante llamadas al sistema (System Calls).
3. **Intercambiabilidad:** Podés cambiar de Shell (usar `bash`, `sh` o incluso interfaces gráficas como XFCE) en cualquier momento sin alterar el núcleo del sistema.

operativo. Si el Shell fuera parte del Kernel, la caída del intérprete de comandos colapsaría la computadora entera.

e.

Sí, es totalmente posible. La teoría indica explícitamente que "cada usuario puede tener una interfaz o shell" definida de manera independiente. Esto se define al momento de crear el usuario (con comandos como `useradd` o `adduser`) o modificando la última columna de la fila del usuario en el archivo de configuración de cuentas `/etc/passwd`. De este modo, un usuario administrativo podría usar `bash`, mientras que una cuenta de servicio de red restringida podría tener asignada una shell limitada o deshabilitada por seguridad (como `/usr/sbin/nologin`).

6. El sistema de Archivos (File System) en Linux:

a. ¿Qué es?

b. ¿Cuál es la estructura básica de los File System en GNU/Linux? Mencione los directorios más importantes e indique qué tipo de información se encuentra en ellos. ¿A qué hace referencia la sigla FHS?

c. Mencione sistemas de archivos soportados por GNU/Linux.

a.

Un sistema de archivos (File System) es un componente de software que se encarga de organizar la forma en que se almacenan y administran los archivos en los dispositivos de almacenamiento físico (como discos rígidos, SSDs o unidades de almacenamiento virtuales). Su propósito es proveer una estructura organizada que permita al sistema operativo y a las aplicaciones crear, leer, escribir, estructurar e identificar carpetas y archivos.

b.

Estructura Básica: A diferencia de otros sistemas operativos que dividen el almacenamiento en letras de unidad físicas (como C:\ o D:\), la estructura básica en GNU/Linux se organiza como un único árbol jerárquico unificado que se expande hacia abajo desde un único punto de origen llamado raíz (/). Además, es una estructura sumamente flexible que permite que cada directorio del sistema pueda estar montado en una partición física de disco diferente (como /home o /tmp) de manera totalmente transparente para el usuario.

Directorios más importantes según el FHS y su contenido:

- **/ (Raíz o Root):** Es el tope de la estructura de directorios (conceptualizado de forma didáctica como el equivalente al C:\ de Windows).
- **/home:** Directorio reservado para almacenar los archivos personales de los usuarios comunes del sistema (como sus documentos, descargas y configuraciones personales).
- **/etc:** Contiene los archivos de configuración de los programas instalados y de los servicios generales del sistema operativo.
- **/bin:** Almacena los archivos binarios y ejecutables esenciales indispensables para el arranque y el funcionamiento básico del sistema, accesibles para todos los usuarios.

- **/var:** Contiene información que varía de tamaño constantemente a medida que se usa la computadora, tales como registros de eventos del sistema o errores (logs), bases de datos y colas de impresión o de correo (*spools*).
- **/dev:** Contiene los enlaces de acceso a los dispositivos de hardware físicos conectados al equipo (como discos, particiones, teclados o puertos USB), lo que responde a la filosofía de Linux de que *"todo es un archivo"*.
- **/usr:** Almacena las aplicaciones y programas de los usuarios junto con sus respectivas librerías de soporte.

La sigla FHS hace referencia a Filesystem Hierarchy Standard (Estándar de Jerarquía del Sistema de Archivos). Es una especificación o norma estándar que define el diseño del árbol de directorios, estableciendo dónde deben ubicarse los archivos y qué tipo de información específica debe contener cada carpeta, garantizando que el sistema sea consistente e independiente de la distribución de Linux que se utilice.

c.

GNU/Linux se caracteriza por ofrecer soporte para una amplia gama de sistemas de archivos (File Systems) de distintos fabricantes, entre los que se destacan:

- **La familia Extended (ext):** Es el sistema de archivos nativo y el adoptado preferentemente por GNU/Linux en la actualidad, abarcando las versiones históricas ext2, ext3 y ext4.
- **Btrfs (B-tree FS):** Desarrollado por Oracle, es un sistema de archivos avanzado que se debate como el reemplazo futuro de la familia ext. Ofrece características de última generación como soporte para archivos de mayor tamaño, mayor tolerancia a fallas, comprobación y reparación de consistencia sin necesidad de desmontar el disco, indexación, snapshots (instantáneas), compresión nativa y desfragmentación integrada.
- **Sistemas de archivos de Windows:** GNU/Linux incluye soporte para leer y escribir en discos formateados con sistemas propietarios como FAT (FAT16, FAT32) y NTFS.
- **Otros sistemas de archivos:** Soporta formatos adicionales como ReiserFS, entre otros.

7. Particiones:

a. Definición. Tipos de particiones. Ventajas y Desventajas.

b. ¿Cómo se identifican las particiones en GNU/Linux? (Considere discos IDE, SCSI y SATA).

c. ¿Cuántas particiones son necesarias como mínimo para instalar GNU/Linux? Nómbralas indicando tipo de partición, identificación, tipo de File System y punto de montaje.

d. Dar ejemplos de diversos casos de particionamiento dependiendo del tipo de tarea que se deba realizar en su sistema operativo.

e. ¿Es posible visualizar particiones del tipo FAT y NTFS (que son de Windows) en GNU/Linux?

f. ¿Qué tipo de software para particionar existe? Menciónelos y compare.

a.

Definición: Una partición es una forma de dividir lógicamente un disco rígido físico. Cada partición se comporta como una unidad de almacenamiento independiente y debe ser formateada con un sistema de archivos de destino (como ext4, FAT o NTFS) para poder almacenar datos.

Tipos de particiones:

- **Partición Primaria:** Es una división directa del disco. Debido a las limitaciones de espacio de la tabla de particiones dentro del MBR (que mide solo 64 bytes y cuenta con 4 entradas de 16 bytes cada una), existe un límite estricto de hasta 4 particiones primarias por disco físico. Su información se almacena directamente en el MBR.
- **Partición Extendida:** Es un tipo especial de partición que sirve exclusivamente como contenedor para alojar particiones lógicas en su interior. Solo puede existir una partición extendida por disco y no se le puede definir un sistema de archivos de manera directa.
- **Partición Lógica:** Se crea dentro del espacio físico de la partición extendida y sí lleva un sistema de archivos definido. El sistema las administra y conecta internamente mediante una lista enlazada.

Ventajas de particionar:

- Permite instalar múltiples sistemas operativos en particiones separadas dentro de un mismo disco.
- Facilita la creación de particiones de restauración o recuperación (*restore*) de todo el sistema.

- Permite colocar el Kernel en una partición configurada como solo lectura, o que ni siquiera se monte tras el arranque, protegiéndolo de accesos de usuarios comunes o software malicioso.

Desventajas de particionar:

- **Particionar en exceso** puede provocar fragmentación del espacio libre, lo que complica la administración y aumenta el riesgo de quedarse sin almacenamiento en una partición crítica (como la raíz /) mientras otra cuenta con espacio de sobra

b.

En GNU/Linux, todos los dispositivos se representan como archivos dentro del directorio /dev. La nomenclatura depende del tipo de interfaz del disco:

- **Discos IDE:** Históricamente se identificaban con el prefijo **hd** seguido de una letra para el disco y un número para la partición (por ejemplo: /dev/hda para el primer disco, /dev/hdb para el segundo).
- **Discos SCSI y SATA:** Se identifican bajo el prefijo **sd** (por ejemplo: /dev/sda para el primer disco físico, /dev/sdb para el segundo).
- **Identificación numérica de las particiones:**
 - Las particiones primarias y extendidas siempre adoptan un número del 1 al 4 al final del disco (ejemplos: /dev/sda1, /dev/sda2).
 - Las particiones lógicas comienzan a numerarse siempre a partir del 5 en adelante (ejemplo: /dev/sda5), sin importar cuántas particiones primarias se hayan creado realmente.

c.

Mínimo absoluto: Es necesaria una sola partición.

Recomendación de la cátedra: Se aconseja crear al menos dos particiones.

Esquema de particionado recomendado (2 particiones):

1. Partición Raíz:

- **Tipo de partición:** Primaria o Lógica.
- **Identificación típica:** /dev/sda1.
- **Tipo de File System:** Extended (versión ext4, que es el nativo y adoptado por GNU/Linux).
- **Punto de montaje:** / (el tope del árbol jerárquico del sistema).

2. Partición de Intercambio (SWAP):

- **Tipo de partición:** Primaria o Lógica.
- **Identificación típica:** /dev/sda2.
- **Tipo de File System:** SWAP (espacio reservado en disco utilizado por el sistema de memoria virtual para auxiliar a la RAM física).
- **Punto de montaje:** **No tiene punto de montaje;** el Kernel la administra de manera directa para realizar el intercambio de páginas de memoria (*swapping*)

d.

Caso 1: Servidor de archivos de red o computadora de escritorio (Desktop)

- **Esquema:** Crear una partición dedicada para /home.
- **Razón:** Almacena de forma separada los documentos de los usuarios. Si el sistema operativo se daña o se actualiza, se puede formatear / e instalar de nuevo sin tocar la partición /home, preservando intactos los archivos personales.

Caso 2: Servidores corporativos de logs, correo o bases de datos

- **Esquema:** Crear una partición separada para el directorio /var.
- **Razón:** Los archivos de log e información variable pueden crecer exponencialmente. Si /var está en una partición aislada y se llena por completo, no afectará a la partición raíz /, permitiendo que el sistema operativo siga funcionando de manera estable.

Caso 3: Entornos de alta seguridad o servidores de misión crítica

- **Esquema:** Crear una partición pequeña para /boot montada como solo lectura.
- **Razón:** Almacena el Kernel de forma segura, impidiendo que usuarios comunes o intrusos modifiquen o corrompan los ejecutables del núcleo del S.O.

Caso 4: Sistemas con memoria RAM limitada

- **Esquema:** Crear una partición de intercambio (SWAP) de tamaño generoso.
- **Razón:** Brinda soporte robusto para la memoria virtual del sistema, evitando cuelgues del S.O. por falta de RAM.

e.

Sí, es totalmente posible. GNU/Linux cuenta con un amplio soporte nativo para diversos sistemas de archivos de otros fabricantes, incluyendo FAT y NTFS. El sistema operativo permite montar estas particiones directamente sobre directorios del sistema de archivos unificado de Linux para leer, escribir y modificar los datos que contengan de manera transparente.

f.

El software para particionar se clasifica en dos tipos básicos:

Criterio	Particionadores Destructivos	Particionadores No Destructivos
Definición	Permiten crear y eliminar particiones, pero no admiten redimensionar o modificar particiones existentes sin destruir los datos contenidos en ellas.	Permiten crear, eliminar y modificar/redimensionar el espacio de particiones activas sin pérdida de datos.
Ejemplos clásicos	fdisk (herramienta interactiva tradicional basada en modo texto).	gparted (interfaz gráfica moderna) o fips.
Uso común	Ideal para preparar discos limpios o configuraciones rápidas por línea de comandos.	Integrados de forma interactiva en la interfaz gráfica de instalación de las distribuciones modernas para redimensionar particiones de Windows sin perder información.

8. Arranque (bootstrap) de un Sistema Operativo:

- a. ¿Qué es el BIOS? ¿Qué tarea realiza?
- b. ¿Qué es UEFI? ¿Cuál es su función?
- c. ¿Qué es el MBR? ¿Qué es el MBC?
- d. ¿A qué hacen referencia las siglas GPT? ¿Qué sustituye? Indique cuál es su formato.
- e. ¿Cuál es la funcionalidad de un “Gestor de Arranque”? ¿Qué tipos existen? ¿Dónde se instalan? Cite gestores de arranque conocidos.
- f. ¿Cuáles son los pasos que se suceden desde que se prende una computadora hasta que el Sistema Operativo es cargado (proceso de bootstrap)?
- g. Analice el proceso de arranque en GNU/Linux y describa sus principales pasos.
- h. ¿Cuáles son los pasos que se suceden en el proceso de parada (shutdown) de GNU/Linux?
- i. ¿Es posible tener en una PC GNU/Linux y otro Sistema Operativo instalado? Justifique.

a.

- **¿Qué es?:** Las siglas hacen referencia a Basic I/O System (Sistema Básico de Entrada/Salida). Es el firmware grabado físicamente de fábrica en un chip de memoria no volátil (como ROM o NVRAM) de la placa madre en arquitecturas x86. En otras arquitecturas cumple un rol equivalente, pero adopta nombres como *OBP (OpenBoot PROM)* en plataformas SPARC o *Power on Reset + IPL* en mainframes.
- **¿Qué tarea realiza?:** Es el responsable de realizar el testeo básico de inicialización del hardware al encender la máquina y hacer disponibles los dispositivos físicos para su uso. Su última acción crítica consiste en buscar, leer y cargar en la memoria principal el código de arranque (MBC) del MBR para transferirle el control de ejecución. Cabe aclarar que el firmware del BIOS tradicional no posee capacidades para leer de forma directa sistemas de archivos.

b.

- **¿Qué es?:** Corresponde a Unified Extensible Firmware Interface (Interfaz de Firmware Extensible Unificada). Es un estándar moderno propiedad del

consorcio *UEFI Forum* (una alianza de compañías líderes como Intel, AMD, Apple, Microsoft, Dell, IBM, entre otras) diseñado con el propósito de modernizar y reemplazar al antiguo BIOS de la década de los 80.

- **¿Cuál es su función?:**
 - Establece una interfaz estandarizada entre el sistema operativo y el firmware del hardware.
 - Define la ubicación exacta de los gestores de arranque y expone información de la configuración del sistema y punteros a rutinas del firmware.
 - Aporta características de última generación que el BIOS no poseía, tales como criptografía y autenticación (como el sistema *Secure Boot* que valida firmas criptográficas de las imágenes para asegurar un arranque libre de código malicioso), soporte para interfaces gráficas interactivas y conexión de red integrada.
 - Provee un BootManager nativo capaz de leer controladores y cargar aplicaciones directamente desde un sistema de archivos compatible (generalmente particiones formateadas en FAT32). Gracias a esto, gestores como GRUB corren como aplicaciones independientes, haciendo innecesaria la tradicional división del arranque en múltiples etapas.

c.

- **MBR (Master Boot Record):** Es el primer sector físico reservado de un disco rígido (ubicado específicamente en el cilindro 0, cabeza 0, sector 1). Su tamaño total es de 512 bytes (el tamaño estándar de un sector físico). Si existen múltiples discos rígidos en el computador, el BIOS requiere que solo uno de ellos sea designado como el disco de arranque principal (*Primary Master Disk*). Estructuralmente está dividido en tres secciones:
 1. El código de arranque inicial o MBC (los primeros 446 bytes).
 2. La tabla de particiones (64 bytes que definen hasta 4 particiones primarias).
 3. La firma de validez de arranque del sector (los últimos 2 bytes).
- **MBC (Master Boot Code):** Es un pequeño programa ejecutable almacenado en el área inicial del MBR (los primeros 446 bytes). Su función es ser el primer código de software que el BIOS lee y ejecuta en memoria para iniciar el sistema operativo. En un sistema configurado, el MBC estándar contiene las instrucciones para invocar al gestor de arranque (*bootloader*).

d.

- **Siglas:** Hacen referencia a GUID Partition Table (Tabla de Particiones con Identificadores Únicos Globales).
- **Qué sustituye:** Sustituye directamente al antiguo esquema de particionado MBR para superar sus limitaciones físicas, como la restricción estricta de un máximo de 4 particiones primarias.
- **Formato de GPT:**
 - Abandona el obsoleto direccionamiento físico de cilindro-cabeza-sector en favor del direccionamiento lógico de bloques (LBA - *Logical Block Addressing*).
 - El bloque LBA 0 mantiene un MBR de protección heredado para asegurar la compatibilidad con sistemas y herramientas antiguas basadas en BIOS.
 - El bloque LBA 1 contiene la cabecera GPT principal, mientras que la tabla de particiones en sí misma se ubica en los bloques lógicos subsiguientes (LBA 2 en adelante).
 - **Cuenta con una estructura de redundancia de datos:** la cabecera de la tabla y los registros de particiones están escritos de manera duplicada tanto al principio como al final físico del disco, protegiendo al sistema ante escrituras corruptas o daños en el soporte.

e.

- **Funcionalidad:** La finalidad del gestor de arranque (o *bootloader*) es la de localizar y cargar la imagen del Kernel (el sistema operativo) en la memoria principal desde su partición correspondiente en el almacenamiento secundario para iniciar su ejecución.
- **Dónde se instalan:** Existen dos métodos principales de instalación física en un disco:
 1. Directamente en el MBR del disco principal (comúnmente ocupando los primeros 446 bytes y utilizando el espacio contiguo del *MBR gap* para fases intermedias).
 2. En el sector de arranque de la partición raíz o partición que esté configurada como activa (conocido como VBR - *Volume Boot Record*).
 3. **Nota:** En plataformas modernas con UEFI, se instala como una aplicación dentro de la partición del sistema de archivos UEFI formateada en FAT32.

- **Gestores conocidos:** Las diapositivas citan a GRUB, LILO, NTLDR, GAG y YaST.
- **Tipos de gestores de arranque:**
 1. **Gestores tradicionales por fases o multietapa (ej. Grub Legacy):** Debido al minúsculo espacio físico del MBR, dividen el arranque en varias fases. La *Fase 1* reside en el MBR y solo se encarga de cargar la *Fase 1.5* (alojada en los 30 KB libres del *MBR gap*), la cual aporta controladores de sistemas de archivos para poder acceder a la *Fase 2* (que despliega la interfaz de usuario en */boot* para seleccionar y cargar el núcleo).
 2. **Gestores de etapa única / Aplicaciones UEFI (ej. Grub 2):** Se ejecutan directamente sobre el firmware de UEFI sin necesidad de etapas complejas, ya que el sistema de archivos es legible de forma nativa por el firmware de la placa.

f.

El proceso de arranque o *bootstrap* se ejecuta a través de una serie de pequeños programas enlazados en cadena:

1. **Encendido / Power-on Reset:** El procesador recibe energía, se inicializa y comienza a ejecutar las primeras instrucciones del firmware.
2. **Inicialización del Firmware (BIOS/UEFI):** El BIOS o la UEFI se ejecutan desde la placa base, analizan la integridad del hardware físico, hacen disponibles los dispositivos de E/S y establecen las configuraciones del sistema.
3. **Lectura del MBR (Fase de BIOS):** Al finalizar sus tareas de diagnóstico, el BIOS busca el disco de arranque principal, lee el Master Boot Code (MBC) del primer sector del disco (MBR), lo carga en la memoria RAM y le cede el control.
4. **Ejecución del Gestor de Arranque:** El MBC invoca y ejecuta el gestor de arranque instalado (por etapas en BIOS o de forma directa en UEFI).
5. **Carga del Kernel:** El gestor de arranque lee del disco la imagen del núcleo del S.O. (Kernel), la copia en la memoria de trabajo y le transfiere el control absoluto al sistema operativo.
6. **Inicialización del Kernel:** El Kernel toma el control, comprueba los dispositivos del equipo y lanza el proceso principal inicial (init).

g.

Una vez que el gestor de arranque inicia el entorno de GNU/Linux, el flujo característico de carga sigue los siguientes pasos:

1. **Carga de Configuración de GRUB:** GRUB se ejecuta y lee su archivo de configuración dinámica (como `/boot/grub/grub.cfg` en Grub 2 o `/boot/grub/menu.lst` en Grub Legacy) para desplegar la pantalla interactiva con la lista de núcleos disponibles.
2. **Traspaso de Parámetros y Carga de Archivos:** Tras la selección, GRUB carga en la RAM el ejecutable del Kernel (como `/vmlinuz-2.6.26`) en modo de lectura y opcionalmente una imagen de sistema de archivos RAM de inicio comprimido (`/initrd-2.6.26.img`), la cual provee los controladores básicos para montar los discos de almacenamiento físico.
3. **Ejecución y Autoconfiguración del Kernel:** El Kernel de Linux toma el control de la CPU en Modo Supervisor, inicializa los controladores de hardware embebidos, monta de manera permanente la partición raíz del sistema (`/`) y libera la memoria temporal del `initrd`.
4. **Lanzamiento de `init` o `systemd`:** El Kernel ejecuta el primer proceso de usuario en el espacio de trabajo con identificador de proceso PID 1, delegando la gestión de servicios y niveles de ejecución a `init` o `systemd`, encargados de inicializar la interfaz de usuario final (consola o escritorio visual XFCE).

h.

1. **Aviso a los usuarios:** Se notifica a los usuarios conectados a la terminal que el sistema se va a detener (por ejemplo, mediante comandos interactivos como `shutdown` de `root`).
2. **Finalización de procesos:** El S.O. envía una señal de terminación (`SIGTERM`) a todas las aplicaciones activas para que guarden sus archivos y cierren de manera ordenada, seguida por una señal de detención forzada (`SIGKILL`) si algún proceso no responde.
3. **Cierre de servicios:** Se apagan en orden inverso los demonios del sistema operativo, servicios de red, colas de impresión y de almacenamiento.
4. **Sincronización y desmontado de discos:** Se escriben todos los datos pendientes en memoria RAM hacia los discos rígidos físicos y se desmontan los sistemas de archivos (o se montan como "solo lectura") para evitar la pérdida o corrupción de datos.
5. **Corte de energía:** Se realiza la llamada al hardware del procesador para detener el reloj del sistema y apagar la fuente de alimentación física.

i.

Sí, es totalmente posible y representa una configuración sumamente común.

- **Justificación por Particionamiento:** El disco físico puede dividirse lógicamente en múltiples particiones de forma que "cada sistema operativo sea instalado en una partición separada". Por ejemplo, en los escenarios de instalación de la cátedra se describe explícitamente una estructura con una partición `/dev/hda1` que contiene DOS con Windows conviviendo de forma simultánea con particiones Linux dedicadas a la raíz `/`, `/boot` y el área de intercambio SWAP.
- **Justificación por el Gestor de Arranque:** Los cargadores modernos como GRUB actúan como gestores de arranque múltiples. Permiten definir un S.O. predeterminado (*default*) y un tiempo de espera de selección (*timeout*), desplegando una interfaz interactiva al encender el equipo para que el usuario elija dinámicamente qué sistema operativo iniciar en cada sesión de trabajo.

9. Archivos y editores

a. ¿Cómo se identifican los archivos en GNU/Linux?

b. Investigue el funcionamiento de los editores vim, nano y mcedit, y los comandos cat, more y less.

c. Cree un archivo llamado "prueba.exe" en su directorio personal usando el vim. El mismo debe contener su número de alumno y su nombre.

d. Investigue el funcionamiento del comando file. Pruébalo con diferentes archivos. ¿Qué diferencia nota?

e. Investigue la funcionalidad y parámetros de los siguientes comandos relacionados con el uso de archivos:

i. cd

ii. mkdir

iii. rmdir

iv. ln

v. tail

vi. locate

vii. ls

viii. pwd

ix. cp

x. mv

xi. find

a.

En GNU/Linux, la identificación y gestión de archivos se rige por principios de diseño fundamentales:

- **Filosofía de "Todo es un archivo":** En este sistema operativo, prácticamente cualquier componente (desde un documento de texto hasta los directorios, las terminales y los dispositivos físicos de hardware como discos o puertos USB) se representa y administra bajo la estructura de un archivo dentro del directorio /dev.
- **Independencia de la extensión:** A diferencia de Windows, donde el sistema operativo utiliza obligatoriamente la extensión (como .exe, .txt o .docx) para saber con qué programa abrir o ejecutar un archivo, en GNU/Linux las extensiones son meramente informativas para el usuario. El sistema operativo

identifica el tipo de archivo analizando su cabecera y contenido interno (utilizando metadatos o "números mágicos").

- **Sensibilidad a las mayúsculas (Case Sensitive):** El sistema distingue estrictamente entre mayúsculas y minúsculas. Por lo tanto, Archivo.txt, archivo.txt y ARCHIVO.TXT son tres archivos completamente distintos y pueden coexistir en la misma carpeta.
- **Ruta Única Jerárquica:** Todos los archivos se identifican de manera unívoca mediante su ruta (*path*) absoluta o relativa, partiendo desde la raíz unificada (/)

b.

Editores de Texto:

1. **vim:** Es un editor de texto en consola sumamente potente y el estándar clásico en sistemas Unix-like. Funciona mediante modos: al abrirlo se inicia en *Modo Comando* (donde cada tecla realiza una acción como borrar o copiar líneas); se debe presionar la tecla i para entrar al *Modo Insertar* y poder escribir texto; y se usa la tecla Esc seguida de comandos como :wq para guardar y salir.
2. **nano:** Es un editor de texto muy sencillo y amigable, ideal para principiantes en la terminal. A diferencia de Vim, no utiliza modos; se puede escribir directamente apenas se abre. En la parte inferior de la pantalla muestra los atajos de teclado principales (donde el símbolo ^ representa la tecla Ctrl, por ejemplo, Ctrl + O para guardar y Ctrl + X para salir).
3. **mcedit:** Es el editor de texto interno que viene integrado en el gestor de archivos visual *Midnight Commander* (mc). Ofrece una interfaz de pantalla completa con menús desplegables e interactivos mediante el uso de las teclas de función (F1 a F10), facilitando la edición rápida sin necesidad de memorizar comandos de consola.

Comandos de Visualización (Paginadores y Lectura):

1. **cat:** Proviene de *concatenate*. Su función principal es mostrar en pantalla el contenido completo de un archivo de un solo tirón. No permite pausar la lectura, por lo que si el archivo es muy largo, el texto pasará de largo velozmente.
2. **more:** Es un paginador interactivo que permite visualizar el contenido de un archivo pantalla por pantalla. En la esquina inferior muestra el porcentaje del archivo leído. Permite avanzar línea por línea (presionando Enter) o pantalla completa (con la barra espaciadora), pero tiene la limitación de que solo permite ir hacia adelante y no se puede retroceder.
3. **less:** Es una versión moderna y muy mejorada de more. No necesita cargar todo el archivo en memoria antes de abrirlo, lo que lo hace increíblemente rápido. Permite navegar de forma libre tanto hacia adelante como hacia atrás usando las flechas del teclado o las teclas Page Up/Page Down. Se sale presionando la tecla q.

d.

El comando file es una herramienta indispensable en GNU/Linux que sirve para determinar el tipo de contenido real de un archivo analizando su estructura y cabeceras internas (mediante pruebas de posición de bytes), sin dejarse engañar por la extensión del archivo.

e.

Comando	Funcionalidad	Parámetros más importantes
cd	Change Directory: Cambia el directorio o carpeta de trabajo actual a la ruta especificada.	cd .. (sube un nivel en el árbol). cd ~ (va al directorio personal o home).
mkdir	Make Directory: Crea uno o varios directorios vacíos en la ruta indicada.	-p: Crea directorios parentales recursivamente si no existen previamente (ej. mkdir -p carpeta1/carpeta2).
rmdir	Remove Directory: Elimina directorios, con la condición estricta de que deben estar completamente vacíos.	-p: Elimina de forma recursiva los directorios padres si al vaciarse también quedan sin contenido.
ln	Link: Crea enlaces o accesos directos entre archivos.	-s: Crea un enlace simbólico (<i>soft link</i> , equivalente al acceso directo de Windows). Si no se usa, crea un enlace duro (<i>hard link</i>).
tail	Muestra por pantalla las últimas líneas de un archivo de texto. Por defecto muestra las últimas 10 líneas.	-n [número]: Especifica exactamente cuántas líneas finales quieres ver (ej. tail -n 5 archivo.txt) -f: Modo dinámico (<i>follow</i>); mantiene la terminal abierta y muestra en tiempo real las líneas que se vayan agregando (ideal para logs en /var/log).
locate	Búsqueda rápida: Busca archivos por nombre en todo el disco mediante una base de datos indexada previamente. Es instantáneo.	Nota: Su limitación es que si creaste un archivo hace un instante, no lo encontrará hasta que actualices el índice del sistema con el comando updatedb (como root).
ls	List: Lista el contenido del directorio actual o de la ruta especificada.	-l: Formato largo detallado (muestra permisos, cantidad de enlaces, propietario, grupo, tamaño en bytes y fecha de modificación). -a: Muestra todos los archivos, incluyendo los archivos ocultos (aquellos cuyo nombre empieza con un punto .).
pwd	Print Working Directory: Imprime en pantalla la ruta absoluta completa del directorio donde estás parado actualmente.	No requiere parámetros usualmente.

cp	Copy: Copia archivos o directorios completos desde un origen a un destino.	-R o -r: Copia directorios completos de forma recursiva (incluyendo todo su contenido interno). -p: Preserva la fecha original, hora y los permisos de acceso de los archivos al copiarlos.
mv	Move: Mueve archivos o directorios de ubicación. También se utiliza para renombrar archivos.	No requiere de parámetros complejos; su sintaxis básica es mv origen destino.
find	Búsqueda exhaustiva: Busca archivos recorriendo físicamente el disco en tiempo real bajo criterios avanzados. Es más lento que locate pero busca en vivo.	-name "[patrón]": Busca archivos por coincidencia de nombre (permite comodines como *). -type [f/d]: Filtra la búsqueda indicando si querés buscar solo archivos (f) o solo directorios (d).

10. Indique qué comando es necesario utilizar para realizar cada una de las siguientes acciones. Investigue su funcionamiento y parámetros más importantes:

- a. Cree la carpeta ISOCSO**
- b. Acceda a la carpeta**
- c. Cree dos archivos con los nombres isocso.txt e isocso.csv**
- d. Liste el contenido del directorio actual**
- e. Visualizar la ruta donde estoy situado**
- f. Busque todos los archivos en los que su nombre contiene la cadena "iso*"**
- g. Informar la cantidad de espacio libre en disco**
- h. Verifique los usuarios conectados al sistema**
- i. Editar a el archivo isocso.txt e ingresar Nombre y Apellido**
- j. Mostrar en pantalla las últimas líneas de un archivo.**

- a. mkdir ISOCSO**
- b. cd ISOCSO**
- c. touch isocso.txt isocso.csv**
- d. ls -l**
- e. pwd**
- f. find . -name "iso*"**
- g. df -h**
- h. who**
- i. vim isocso.txt**
- j. tail isocso.txt**

11. Investigue el objetivo, parámetros y ubicación (directorio) de los siguientes comandos:

→ man

→ shutdown

→ reboot

→ halt

→ uname

→ dmesg

→ lspci

→ at

→ head

→ tail

→ man:

- **Objetivo:** Interfaz para acceder a los manuales de referencia y documentación del sistema. Permite buscar y leer explicaciones detalladas sobre comandos de usuario, archivos de configuración y llamadas al sistema (*system calls*).
- **Parámetros más importantes:**
 - **[sección] [comando]:** Permite especificar el área del manual (por ejemplo, man 2 write abre la sección 2 correspondiente a llamadas al sistema, mientras que man exit abre la documentación estándar del comando).
 - **-k [palabra_clave]:** (Equivalente a apropos). Busca la palabra clave en las descripciones breves de todos los manuales del sistema.
- **Ubicación (Path):** /usr/bin/man (Ubicado en /usr/bin según el FHS por ser una aplicación interactiva del usuario).

→ shutdown:

- **Objetivo:** Permite apagar, reiniciar o detener la computadora de forma segura y planificada, enviando advertencias a los terminales de los usuarios que estén conectados.

- **Parámetros más importantes:**
 - **-h (o --poweroff):** Apaga y corta la energía eléctrica del equipo.
 - **-r (o --reboot):** Ordena reiniciar el sistema una vez finalizado el apagado ordenado.
 - **[tiempo]:** Especifica cuándo realizar la acción (por ejemplo, now para inmediato, o un tiempo relativo como +15 para dentro de 15 minutos).
 - **"[mensaje_de_advertencia]":** Envía una notificación de texto personalizada a todas las terminales activas antes del apagado.
- **Ubicación (Path):** /usr/sbin/shutdown (Históricamente /sbin/shutdown). Se almacena en los directorios de binarios de administración (*System Binaries*) dado que es de uso exclusivo o prioritario para el superusuario root.

→ reboot:

- **Objetivo:** Detiene el sistema operativo de forma inmediata y le envía la orden de reiniciar a la CPU.
- **Parámetros más importantes:**
 - **-f (force):** Fuerza el reinicio inmediato del hardware de manera drástica, omitiendo el apagado limpio que normalmente gestiona el sistema de servicios.
 - **-p (poweroff):** Intenta apagar el equipo en lugar de reiniciarlo.
- **Ubicación (Path):** /usr/sbin/reboot (Históricamente /sbin/reboot), clasificado como binario administrativo para mantenimiento del sistema.

→ halt:

- **Objetivo:** Detiene la CPU y suspende toda la ejecución de procesos de manera segura, dejando la máquina en un estado inactivo para que pueda apagarse manualmente sin peligro de corromper el disco físico.
- **Parámetros más importantes:**
 - **-p:** Apaga físicamente la fuente de alimentación (realiza un *poweroff*) tras detener las tareas de la CPU.
 - **-f:** Detiene el sistema de forma abrupta y forzada sin pasar por los scripts de limpieza ordenados del sistema operativo.
- **Ubicación (Path):** /usr/sbin/halt (Históricamente /sbin/halt), dentro del grupo de utilidades del sistema para administración de hardware.

→ **uname:**

- **Objetivo:** Imprime en pantalla información variada y metadatos sobre el sistema operativo, el hardware de la máquina y la versión del núcleo (Kernel) en ejecución.
- **Parámetros más importantes:**
 - **-a (all):** Muestra absolutamente toda la información del sistema en una sola línea (nombre del Kernel, nombre de red del equipo, versión, arquitectura de hardware, etc.).
 - **-r (release):** Muestra de manera aislada la versión de lanzamiento del Kernel actual (útil para contrastar con las imágenes guardadas en /boot).
 - **-m:** Muestra la arquitectura física del procesador (por ejemplo, x86_64 o i686).
- **Ubicación (Path):** /usr/bin/uname (Históricamente /bin/uname). Al ser una consulta de información de acceso público para cualquier tipo de usuario, se ubica en el directorio de binarios de usuario.

→ **dmesg:**

- **Objetivo:** Permite visualizar y controlar el búfer de mensajes de diagnóstico del Kernel. Muestra los eventos de arranque, la detección inicial de hardware físico y la inicialización de controladores de dispositivos (*drivers*).
- **Parámetros más importantes:**
 - **-T (human-readable):** Traduce las marcas de tiempo brutas del Kernel a un formato de fecha y hora legible para los seres humanos.
 - **-l [nivel] (level):** Filtra los mensajes por su gravedad (por ejemplo, -l err,warn muestra únicamente errores y advertencias del hardware).
 - **-w (follow):** Mantiene la terminal abierta de forma interactiva y escribe en pantalla los nuevos eventos del Kernel a medida que ocurren.
- **Ubicación (Path):** /usr/bin/dmesg (Históricamente /bin/dmesg), accesible para que todos los usuarios puedan verificar alertas de dispositivos.

→ **lspci:**

- **Objetivo:** Lista detalladamente todos los dispositivos, adaptadores y controladores de hardware conectados a los puertos e interfaces PCI y PCI Express de la placa madre (tarjetas gráficas, placas de red, adaptadores de sonido, etc.).

- **Parámetros más importantes:**
 - **-v (verbose):** Muestra información técnica detallada de cada dispositivo PCI (direcciones de E/S, buffers de memoria, IRQ).
 - **-t (tree):** Dibuja un esquema de árbol jerárquico que describe cómo están conectados los dispositivos entre los diferentes puentes y buses del computador.
- **Ubicación (Path):** /usr/bin/lspci (en algunas distribuciones puede alojarse en /usr/sbin/lspci como herramienta de administración).

→ **at:**

- **Objetivo:** Permite programar la ejecución diferida de una secuencia de comandos o tareas para que se realicen por única vez en un momento específico del futuro.
- **Parámetros más importantes:**
 - **[tiempo]:** Especifica el horario o fecha de ejecución (por ejemplo, at 08:30 PM, at teatime, o at now + 3 days).
 - **-f [archivo]:** Lee los comandos de automatización desde el archivo de texto especificado en lugar de esperarlos por el teclado.
 - **-l (o comando directo atq):** Lista la fila de trabajos que se encuentran pendientes de ejecución programada en el sistema.
 - **-d [ID_tarea] (o comando directo atrm):** Remueve de la cola y cancela la tarea programada asociada a ese número de ID.
- **Ubicación (Path):** /usr/bin/at, una aplicación de usuario estándar para la automatización de tareas programadas.

→ **head:**

- **Objetivo:** Muestra en pantalla de forma rápida la parte inicial (la cabecera) de uno o varios archivos de texto, evitando tener que abrir un editor completo.
- **Parámetros más importantes:**
 - **-n [número]:** Especifica la cantidad exacta de líneas que querés imprimir partiendo desde la primera (por defecto muestra las primeras 10 líneas si no se le indica lo contrario).
 - **-c [bytes]:** Muestra los primeros caracteres del archivo hasta completar la cantidad de bytes indicada.

- **Ubicación (Path):** /usr/bin/head (Históricamente /bin/head), comando esencial de lectura de archivos accesible para todos los usuarios.

→ **tail:**

- **Objetivo:** Muestra en pantalla la parte final (la cola) de uno o varios archivos de texto. Es la herramienta por excelencia para revisar y monitorear archivos que registran eventos constantes en el sistema.
- **Parámetros más importantes:**
 - **-n [número]:** Especifica la cantidad de líneas que querés ver desde el final del archivo (por defecto muestra las últimas 10 líneas).
 - **-f (*follow*):** Modo dinámico o de seguimiento activo; no cierra la terminal al leer el final del archivo, sino que lo mantiene abierto y escribe en pantalla en tiempo real las líneas que otros procesos del sistema le vayan anexando (esencial para analizar los logs interactivos de la carpeta /var/log).
- **Ubicación (Path):** /usr/bin/tail (Históricamente /bin/tail), binario estándar del sistema para visualización de textos.

12. Proceso de Arranque SystemV :

- a. Enumere los pasos del proceso de inicio de un sistema GNU/Linux, desde que se prende la PC hasta que se logra obtener el login en el sistema.
- b. Proceso INIT. ¿Quién lo ejecuta? ¿Cuál es su objetivo?
- c. RunLevels. ¿Qué son? ¿Cuál es su objetivo?
- d. ¿A qué hace referencia cada nivel de ejecución según el estándar? ¿Dónde se define qué Runlevel ejecutar al iniciar el sistema operativo? ¿Todas las distribuciones respetan estos estándares?
- e. Archivo /etc/inittab. ¿Cuál es su finalidad? ¿Qué tipo de información se almacena en él? ¿Cuál es la estructura de la información que en él se almacena?
- f. Suponga que se encuentra en el runlevel . Indique qué comando(s) deberá ejecutar para cambiar al runlevel . ¿Este cambio es permanente? ¿Por qué?
- g. Scripts RC. ¿Cuál es su finalidad? ¿Dónde se almacenan? Cuando un sistema GNU/Linux arranca o se detiene se ejecutan scripts, indique cómo determina qué script ejecutar ante cada acción. ¿Existe un orden para llamarlos? Justifique.

a.

1. **Fases de Arranque y el rol de init (PID 1):** El flujo va desde el autotest del hardware (BIOS/UEFI) y el cargador de arranque (GRUB) hasta la ejecución de /sbin/init por el Kernel, el cual lee /etc/inittab y lanza la inicialización global del sistema operativo.
2. **Definiciones y diferencias de Runlevels (0-6):** Existen siete niveles de operación estándar que permiten conmutar estados del sistema (desde apagado hasta entorno gráfico). No obstante, distribuciones como Debian y Red Hat implementaron estos niveles históricamente de forma diferente.
3. **Estructura del archivo /etc/inittab:** Se organiza mediante registros id:runlevels:action:process, controlando el comportamiento detallado del arranque, la ejecución síncrona o en background de procesos y el reinicio automático de terminales de inicio de sesión (getty).
4. **Scripts RC, prefijos 'K'/'S' y orden de dependencias:** Los servicios de cada runlevel se gestionan en directorios /etc/rcX.d/ usando enlaces simbólicos hacia /etc/init.d/. El orden de inicio o parada es estrictamente alfanumérico secuencial para respetar la dependencia indispensable entre servicios (por ejemplo, iniciar la red antes de servidores web).
5. **Transiciones temporales y permanentes:** Los comandos como telinit o init permiten un cambio de estado temporal en memoria, mientras que para un

cambio persistente es necesario editar la directiva `initdefault` dentro del archivo físico de configuración.

b.

- **¿Quién lo ejecuta?:** El proceso **init** es ejecutado directamente por el Kernel (núcleo) de Linux en la última fase del proceso de arranque (*bootstrap*). El Kernel busca el ejecutable del inicializador en rutas fijas estándar como `/sbin/init`, `/etc/init`, `/bin/init` o `/bin/sh` y le transfiere el control. Si el Kernel no logra localizar este programa en el disco, se genera una falla crítica de sistema conocida como *Kernel Panic*.
- **¿Cuál es su objetivo?:** Es el primer proceso de usuario en ejecutarse en el espacio de trabajo, adoptando de forma exclusiva el identificador PID 1 (Process ID 1) y convirtiéndose en el padre de todos los procesos del sistema operativo. Su objetivo primordial es inicializar, organizar y coordinar el espacio de usuario, lanzando los servicios y demonios necesarios para que la computadora quede operativa y lista para el inicio de sesión.

c.

- **¿Qué son?:** Los Runlevels (o niveles de ejecución) son estados lógicos de funcionamiento en los que se puede encontrar un sistema operativo basado en SysV init, representados tradicionalmente por números que van del 0 al 6.
- **¿Cuál es su objetivo?:** Su propósito consiste en definir qué servicios de software y procesos de fondo (daemons) deben estar activos o inactivos en cada estado del sistema, permitiendo al administrador de sistemas conmutar de forma segura entre diferentes modos de operación (desde un modo mínimo de reparación sin red hasta un entorno multiusuario completo con entorno gráfico).

d.

Niveles según el estándar tradicional:

- **0: Apagado (Halt / Poweroff):** Se detienen todos los servicios de forma ordenada y se apaga físicamente la fuente de alimentación del equipo.
- **1 (o S): Modo Monousuario (Single-user):** Reservado para tareas de mantenimiento y reparación del sistema. Solo se permite el acceso al

superusuario root en una sola consola de texto y sin servicios de red activos.

- **2: Modo Multiusuario sin soporte de red** (en sistemas de tipo Red Hat).
 - **3: Modo Multiusuario completo en consola de texto** (con red habilitada, pero sin interfaz gráfica).
 - **4: No usado / Personalizable:** Reservado para configuraciones especiales que el administrador de sistemas desee definir.
 - **5: Modo Multiusuario completo con entorno gráfico (GUI):** Levanta el servidor gráfico y los gestores de pantalla para el inicio de sesión visual.
 - **6: Reinicio (Reboot):** Apaga todos los servicios de manera controlada y ordena a la CPU reiniciar el hardware.
- **Dónde se define:** El nivel de ejecución predeterminado en el inicio se define en el archivo de configuración `/etc/inittab` mediante la directiva `initdefault` (por ejemplo: `id:3: initdefault:` para iniciar en consola o `id:5:initdefault:` para interfaz gráfica).
 - **¿Todas las distribuciones respetan estos estándares?:** No. Existen discrepancias notables según la distribución. Mientras que la familia Red Hat (RHEL, Fedora) separa estrictamente el nivel 3 (consola con red) del nivel 5 (entorno gráfico), distribuciones como Debian configuran tradicionalmente los runlevels del 2 al 5 de forma idéntica, iniciando de manera predeterminada en el runlevel 2 con el entorno gráfico y la red completamente habilitados de fábrica.

e.

- **Finalidad:** Es el archivo de configuración principal del proceso `init`. Su propósito es indicarle al sistema qué scripts y programas debe lanzar al arrancar, qué hacer cuando se presiona la combinación de teclas `Ctrl+Alt+Del` o ante fallos de energía, y qué terminales de inicio de sesión (`getty`) debe mantener activas en la pantalla de la PC.
- **Información almacenada:** Almacena directivas lógicas secuenciales que determinan el comportamiento detallado del arranque, la ejecución síncrona o asíncrona de scripts y el reinicio automático de procesos críticos.
- **Estructura de la información:** Cada línea útil del archivo `/etc/inittab` sigue un formato estructurado de cuatro campos separados por dos puntos:

`id:runlevels:action:process`

- **id:** Un identificador único de 1 a 4 caracteres para referenciar la directiva.
- **runlevels:** El número o lista de runlevels en los cuales se debe aplicar la directiva (por ejemplo, 35 para ejecutarla solo en los niveles 3 y 5; si está vacío, se aplica a todos).
- **action:** Define cómo debe comportarse init con respecto al proceso. Las acciones clave son:
 - **initdefault:** Indica el runlevel inicial por defecto al bootear.
 - **sysinit:** Ejecuta el script antes de cualquier otra tarea (normalmente lanza /etc/init.d/rcS o /etc/rc.sysinit).
 - **wait:** Lanza el proceso y espera a que termine antes de continuar con la siguiente línea de inittab.
 - **respawn:** Ejecuta el programa y, si este finaliza o se cae, init lo vuelve a iniciar automáticamente (esencial para mantener vivas las consolas de login).
- **process:** El comando ejecutable o script de shell real con sus respectivos argumentos de sistema.

f.

- **Comandos para cambiar de runlevel:** Para conmutar el estado del sistema en caliente, el superusuario root debe ejecutar el comando `init <Y>` o su equivalente `telinit <Y>` (por ejemplo, `init 3` para pasar de modo gráfico a modo consola).
- **¿Es permanente?: No, este cambio es puramente temporal.** El cambio de nivel se realiza únicamente en la memoria RAM de la sesión actual de trabajo. En cuanto la computadora se reinicie, el proceso init volverá a leer el archivo físico de configuración `/etc/inittab` y cargará el runlevel predeterminado definido en la línea `initdefault`. Para hacerlo persistente, se debe editar el archivo `/etc/inittab` con un editor de texto (como vim o nano) y cambiar el número del runlevel en dicha directiva.

g.

- **Comandos para cambiar de runlevel:** Para conmutar el estado del sistema en caliente, el superusuario root debe ejecutar el comando `init <Y>` o su

equivalente telinit <Y> (por ejemplo, init 3 para pasar de modo gráfico a modo consola).

- **¿Es permanente?:** No, este cambio es puramente temporal. El cambio de nivel se realiza únicamente en la memoria RAM de la sesión actual de trabajo. En cuanto la computadora se reinicie, el proceso init volverá a leer el archivo físico de configuración /etc/inittab y cargará el runlevel predeterminado definido en la línea initdefault. Para hacerlo persistente, se debe editar el archivo /etc/inittab con un editor de texto (como vim o nano) y cambiar el número del runlevel en dicha directiva.

¿Existe un orden de ejecución? Sí, se ejecutan en estricto orden alfanumérico. Los enlaces simbólicos se nombran bajo el formato [K|S][Número_de_dos_dígitos][Nombre_del_Servicio] (por ejemplo: S10network, S20ssh, S91apache). El número de dos dígitos establece la secuencia de lanzamiento. Este orden secuencial es indispensable debido a las dependencias de servicios: no sería posible iniciar un servidor web Apache (S91apache) de manera exitosa si previamente no se hubiera inicializado el soporte de red física (S10network) y el soporte de almacenamiento en disco, garantizando de esta manera la estabilidad y consistencia lógica del sistema.

13. SystemD (<https://github.com/systemd/systemd>):

a. ¿Qué es systemd?

b. ¿A qué hace referencia el concepto de Unit en SystemD?

c. ¿Para qué sirve el comando systemctl en SystemD?

d. ¿A qué hace referencia el concepto de target en SystemD?

e. Ejecutar el comando pstree. ¿Qué es lo que se puede observar a partir de la ejecución de este comando?

a.

Systemd es un sistema de iniciación y un gestor de sistema y de servicios (init system) moderno diseñado exclusivamente para sistemas operativos GNU/Linux. Fue creado para reemplazar por completo al antiguo y secuencial sistema de inicio clásico SysV init.

Al igual que su predecesor, se ejecuta inmediatamente después de que el Kernel de Linux se inicializa, adoptando de forma exclusiva el PID 1 (Process ID 1) para convertirse en el padre de todos los procesos del espacio de usuario.

Su gran diferencia y ventaja reside en que está diseñado bajo una arquitectura de paralelismo masivo. Mientras que SysV init inicia los servicios de forma estrictamente secuencial (uno por uno), systemd los arranca en forma simultánea utilizando la activación por sockets (socket-based activation) y buses de mensajería (D-Bus), lo que optimiza y reduce drásticamente el tiempo total de arranque de la computadora.

b.

En systemd, una Unit (unidad) es el bloque de construcción u objeto de configuración fundamental del sistema. En lugar de utilizar los largos y complejos scripts de consola de SystemV, systemd abstrae y administra cada recurso del sistema mediante archivos declarativos de configuración estructurada llamados *unidades* (los cuales llevan un sufijo de archivo que indica su tipo).

Los tipos de Units más importantes que gestiona el sistema son:

- **service:** Define un demonio o servicio de fondo que puede ser iniciado, detenido, reiniciado o recargado (ejemplos comunes: ssh.service, apache2.service).

- **socket:** Modela sockets de red (IP) o de archivos locales (UNIX sockets) para habilitar el inicio automático de un servicio en cuanto llega una conexión de red.
- **device:** Representa dispositivos físicos de hardware detectados y expuestos por el Kernel de Linux (integrado con el gestor udev).
- **mount y automount:** Controla los puntos de montaje de los discos y sistemas de archivos (reemplazando y automatizando de forma flexible el archivo tradicional /etc/fstab).
- **timer:** Unidades que sirven para programar tareas basándose en el tiempo o eventos, actuando como un reemplazo directo o complemento moderno al demonio tradicional cron.
- **target:** Unidades lógicas especiales que sirven para agrupar y coordinar otras unidades durante el inicio.

c.

El comando `systemctl` es la herramienta de línea de comandos principal que el administrador del sistema operativo utiliza para controlar, inspeccionar e interactuar con el gestor `systemd`. Reemplaza y unifica las antiguas herramientas de SystemV como `service`, `chkconfig` y `update-rc.d`.

Las funciones básicas de `systemctl` que vas a utilizar en tu máquina virtual Debian son:

- **Controlar servicios en caliente (tiempo real):**
 - `systemctl start [servicio]` (Arranca el servicio).
 - `systemctl stop [servicio]` (Detiene el servicio).
 - `systemctl restart [servicio]` (Reinicia el servicio).
 - `systemctl status [servicio]` (Muestra información detallada sobre si está activo, su PID, su consumo y sus últimos logs).
- **Administrar el arranque automático (persistente):**
 - `systemctl enable [servicio]` (Crea los enlaces simbólicos para que el servicio se inicie automáticamente en cada arranque de la máquina).
 - `systemctl disable [servicio]` (Deshabilita el arranque automático).
- **Mantenimiento del gestor:**

- `systemctl daemon-reload` (Ordena a `systemd` que vuelva a leer y aplicar los cambios realizados en los archivos físicos de configuración de las unidades).

d.

Un **target** (objetivo) es una unidad especial de configuración (con el sufijo `.target`) cuya finalidad es la de agrupar de manera lógica e interconectar múltiples unidades del sistema para establecer un estado u objetivo de funcionamiento.

Cumplen el rol equivalente a los antiguos niveles de ejecución (**Runlevels**) de **SystemV**, pero con una diferencia clave: mientras que en **SysV** los **runlevels** eran estrictamente números excluyentes (0 al 6), los **targets** de `systemd` se identifican mediante nombres descriptivos y pueden estar activos varios de ellos de manera simultánea en forma de un grafo de dependencias dinámico.

Las correspondencias históricas más comunes son:

- **poweroff.target**: Equivale al **Runlevel 0** (Apagado limpio del sistema).
- **rescue.target**: Equivale al **Runlevel 1 / S** (Modo de rescate monousuario).
- **multi-user.target**: Equivale al **Runlevel 3** (Modo consola multiusuario completo con soporte de red activa).
- **graphical.target**: Equivale al **Runlevel 5** (Modo multiusuario completo con red y entorno gráfico activo).
- **reboot.target**: Equivale al **Runlevel 6** (Reinicio de la computadora).

e.

El comando `ps-tree` (*Process Tree* o *Árbol de Procesos*) es una utilidad interactiva de consola que permite visualizar de manera gráfica todos los procesos que están corriendo en memoria en forma de una estructura de árbol jerárquico.

Al ejecutarlo en la consola, pude observar lo siguiente:

1. **La Raíz Suprema (PID 1)**: Vas a ver que el proceso inicial que encabeza el árbol en el extremo izquierdo de la pantalla es `systemd` (o `init` según la configuración de tu sistema). Esto demuestra de forma visual la teoría de la cátedra de que este es el proceso padre indiscutible del espacio de usuario.
2. **Relación Padre-Hijo (Parent-Child)**: El árbol te muestra qué procesos específicos crearon a otros. Por ejemplo, vas a poder ver de forma clara que tu

consola terminal (bash) es hija de un servicio de entorno gráfico o un gestor de terminales, y que si ejecutas un comando de búsqueda, este nacerá temporalmente como hijo de bash.

3. **Hilos de Ejecución (Threads):** Los subprocesos o hilos livianos de ejecución creados por programas complejos (como navegadores web o gestores de bases de datos) se visualizan agrupados o encerrados entre llaves {} o ramificaciones compactas de su proceso principal.

14. Usuarios:

a. ¿Qué archivos son utilizados en un sistema GNU/Linux para guardar la información de los usuarios?

b. ¿A qué hacen referencia las siglas UID y GID? ¿Pueden coexistir UIDs iguales en un sistema GNU/Linux? Justifique.

c. ¿Qué es el usuario root? ¿Puede existir más de un usuario con este perfil en GNU/Linux? ¿Cuál es la UID de root?

d. Agregue un nuevo usuario llamado isocso a su instalación de GNU/Linux, especifique que su home sea creada en /home/isocso, y hágalo miembro del grupo informatica (si no existe, deberá crearlo). Luego, sin iniciar sesión como este usuario cree un archivo en su home personal que le pertenezca. Luego de todo esto, borre el usuario y verifique que no queden registros de él en los archivos de información de los usuarios y grupos.

e. Investigue la funcionalidad y parámetros de los siguientes comandos:

i. useradd y adduser

ii. usermod

iii. userdel

iv. su

v. groupadd

vi. who

vii. groupdel

viii. passwd

a.

En GNU/Linux, toda la configuración del sistema reside en el directorio /etc. La información sobre las cuentas de usuario y los grupos de seguridad se almacena en cuatro archivos de texto plano fundamentales:

1. **/etc/passwd:** Almacena el registro básico de las cuentas de usuario del sistema. Cada línea representa un usuario y contiene siete campos separados por dos puntos (:):

- **Nombre de usuario (login).**

- **Marcador de contraseña** (históricamente la clave cifrada, hoy en día una x indicando que se delega al archivo *shadow*).
 - **Identificador de usuario (UID).**
 - **Identificador de grupo primario (GID).**
 - **Información de contacto o nombre real (GECOS).**
 - **Directorio personal (home).**
 - **Intérprete de comandos por defecto (Shell / CLI).**
2. **/etc/shadow:** Almacena de manera cifrada y segura las contraseñas de los usuarios. Por razones críticas de seguridad, este archivo posee permisos sumamente restrictivos (solo legible por el superusuario root) para evitar ataques de fuerza bruta o descifrado. Adicionalmente, guarda las directivas de caducidad e inactividad de las claves.
 3. **/etc/group:** Contiene la definición de los grupos de seguridad del sistema operativo. Su estructura define el nombre del grupo, el identificador GID y la lista de usuarios que pertenecen de forma secundaria a dicho grupo.
 4. **/etc/gshadow:** Equivalente a /etc/shadow pero destinado a almacenar las contraseñas cifradas opcionales y los administradores de cada grupo de seguridad.

b.

- **UID (User Identifier):** Es un valor numérico entero y único que el Kernel de GNU/Linux asocia de manera interna a cada cuenta de usuario para identificarla y controlar sus permisos sobre los archivos y procesos.
- **GID (Group Identifier):** Es el identificador numérico que se asigna a cada grupo del sistema para organizar y catalogar la pertenencia y privilegios colectivos de los usuarios.
- **¿Pueden coexistir UIDs iguales?:**
 - *Desde la perspectiva del sistema operativo:* Sí, técnicamente es posible editar a mano el archivo /etc/passwd y asignarle el mismo número de UID a dos nombres de usuario distintos (por ejemplo, crear una cuenta auditor con UID 0).
 - *Justificación técnica de por qué NO debe hacerse:* En GNU/Linux, el Kernel no reconoce nombres de texto, sino únicamente los números de UID. Si dos usuarios comparten el mismo UID, el sistema de archivos y el

planificador de procesos los considerará la misma e idéntica persona física. Esto destruye el principio de seguridad de *no repudio* (no se puede auditar quién realizó realmente una acción en el sistema), provoca un colapso en la privacidad de los directorios personales de cada uno y expone al sistema a severas fallas de seguridad.

c.

- **¿Qué es el usuario root?:** Es la cuenta de superusuario o administrador del sistema operativo. Tiene privilegios absolutos e irrestrictos sobre todo el software y hardware, pudiendo modificar cualquier archivo de configuración (incluso de otros usuarios o del propio Kernel), lanzar y matar cualquier proceso, y alterar permisos de manera global.
- **¿Puede existir más de un usuario con este perfil?:** Sí. Como se explicó en el inciso anterior, si se le asigna el UID 0 a cualquier otra cuenta en `/etc/passwd`, ese usuario obtendrá de forma automática e inmediata los mismos privilegios totales de administración que tiene root. Sin embargo, esto es considerado una pésima práctica de seguridad. En su lugar, se utilizan herramientas como `sudo` para delegar tareas específicas a usuarios comunes temporalmente.
- **¿Cuál es la UID de root?:** El identificador UID asignado de forma universal e inamovible al usuario root es 0.

e.

i. `useradd` y `adduser`

- **useradd:** Es una utilidad de bajo nivel nativa de UNIX para la creación de usuarios. Es un binario directo.
 - **Parámetros importantes:**
 - **-m:** Crea automáticamente el directorio personal del usuario (`/home/usuario`) si no existe.
 - **-d [ruta]:** Especifica una ruta personalizada para el directorio de inicio del usuario.
 - **-g [grupo]:** Define el grupo primario del usuario (debe existir previamente).
 - **-G [grupo1,grupo2]:** Define los grupos secundarios o adicionales a los que pertenecerá.

- **-s [shell]:** Asigna el intérprete de comandos por defecto (ej. /bin/bash).
- **adduser:** Es un script amigable de alto nivel (escrito en Perl) muy común en Debian. A diferencia de useradd, este comando es interactivo: va pidiendo al administrador la contraseña, el nombre real, el teléfono y otros datos directamente en la consola, y crea el directorio home y los archivos de entorno iniciales (/etc/skel) de forma totalmente automática.

ii. usermod

- **Funcionalidad:** Permite modificar de forma dinámica las propiedades y atributos existentes de una cuenta de usuario ya creada.
- **Parámetros importantes:**
 - **-l [nuevo_nombre]:** Cambia el nombre de inicio de sesión (*login*) del usuario.
 - **-d [nueva_ruta] -m:** Mueve el contenido del directorio home anterior a su nueva ubicación física.
 - **-g [grupo]:** Modifica el grupo primario.
 - **-a -G [grupo]:** Agrega (append) un grupo secundario al usuario sin borrar los grupos secundarios que ya tenía asignados previamente (parámetro sumamente crítico, ya que omitir el -a quita al usuario de todos sus otros grupos).
 - **-L (Lock) / -U (Unlock):** Bloquea o desbloquea temporalmente la cuenta del usuario impidiéndole iniciar sesión en el sistema.

iii. userdel

- **Funcionalidad:** Elimina una cuenta de usuario del sistema (remueve sus registros de /etc/passwd y /etc/shadow).
- **Parámetros importantes:**
 - **-r (recursive):** Elimina también de forma física el directorio home del usuario y su cola de correo interna, evitando dejar archivos huérfanos sin propietario en el disco.
 - **-f (force):** Fuerza la eliminación de la cuenta incluso si el usuario tiene una sesión activa abierta en ese momento.

iv. su

- **Funcionalidad:** Permite sustituir o cambiar la identidad de la terminal actual por la de otro usuario del sistema (generalmente root) sin necesidad de cerrar la sesión activa.
- **Parámetros importantes:**
 - **- (o -l):** Inicia sesión emulando un arranque completo (*login shell*). Carga el entorno de trabajo, las variables, el PATH de administración y se posiciona en el directorio home del nuevo usuario de manera limpia.
 - **-c [comando]:** Ejecuta un único comando con los privilegios del otro usuario y vuelve inmediatamente a la terminal del usuario común sin mantener abierta la sesión del superusuario.

v. groupadd

- **Funcionalidad:** Crea un nuevo grupo de seguridad en el sistema operativo (agregando el correspondiente registro en /etc/group).
- **Parámetros importantes:**
 - **-g [GID]:** Permite especificar un número GID personalizado para el grupo (si no se usa, el sistema asigna el siguiente número libre disponible).
 - **-r:** Crea un grupo especial del sistema (con un GID menor a 1000, reservado para servicios del sistema operativo).

vi. who

- **Funcionalidad:** Imprime en pantalla una lista de todos los usuarios que tienen actualmente una sesión abierta en las terminales físicas o virtuales de la máquina.
- **Parámetros importantes:**
 - **-a (all):** Muestra información sumamente detallada, incluyendo el tiempo de inactividad, los procesos de inicio del sistema y los cambios en el reloj.
 - **-r:** Muestra el nivel de ejecución (*Runlevel*) actual en el que se encuentra trabajando el sistema operativo.

vii. groupdel

- **Funcionalidad:** Elimina de forma definitiva un grupo de seguridad del sistema operativo.
- **Nota:** No permite eliminar el grupo primario de un usuario activo en el sistema.

viii. passwd

- **Funcionalidad:** Permite al administrador o a un usuario común establecer, cambiar o actualizar la contraseña de acceso registrada en `/etc/shadow`.
- **Parámetros importantes:**
 - **[nombre_usuario]:** Si lo ejecuta root, puede cambiar la clave de cualquier usuario sin necesidad de conocer la clave anterior. Un usuario común solo puede ejecutar `passwd` a secas para cambiar su propia clave.
 - **-d (delete):** Elimina la clave del usuario (permite iniciar sesión sin ingresar contraseña, de uso exclusivo para root).
 - **-l (lock) / -u (unlock):** Bloquea o habilita la contraseña de una cuenta de usuario de manera segura.

15. FileSystem y permisos:

- a. ¿Cómo son definidos los permisos sobre archivos en un sistema GNU/Linux?
- b. Investigue la funcionalidad y parámetros de los siguientes comandos relacionados con los permisos en GNU/Linux:
 - i. chmod
 - ii. chown
 - iii. chgrp
- c. Al utilizar el comando chmod generalmente se utiliza una notación octal asociada para definir permisos. ¿Qué significa esto? ¿A qué hace referencia cada valor?
- d. ¿Existe la posibilidad de que algún usuario del sistema pueda acceder a determinado archivo para el cual no posee permisos? Indíquelo y realice las pruebas correspondientes.
- e. Explique los conceptos de “full path name” (path absoluto) y “relative path name” (path relativo). De ejemplos claros de cada uno de ellos.
- f. ¿Con qué comando puede determinar en qué directorio se encuentra actualmente? ¿Existe alguna forma de ingresar a su directorio personal sin necesidad de escribir todo el path completo? ¿Podría utilizar la misma idea para acceder a otros directorios? ¿Cómo? Explique con un ejemplo.
- g. Investigue la funcionalidad y parámetros de los siguientes comandos relacionados con el uso del FileSystem:
 - i. umount
 - ii. du
 - iii. df
 - iv. mount
 - v. mkfs
 - vi. fdisk (con cuidado)
 - vii. write
 - viii. losetup
 - ix. sta

a.

En GNU/Linux, bajo la premisa de diseño de que "todo es un archivo" (incluyendo directorios y dispositivos de hardware), el sistema operativo implementa un esquema de seguridad de acceso basado en tres categorías de usuarios y tres tipos de acciones:

1. **Categorías de Usuarios:**

- **Dueño/Propietario (u - User):** El usuario que creó el archivo o posee su propiedad legal en el sistema.
- **Grupo (g - Group):** Un conjunto de usuarios agrupados que comparten el mismo nivel de acceso colectivo al archivo.
- **Otros (o - Others):** Cualquier otra cuenta de usuario creada en el sistema que no sea el dueño ni pertenezca al grupo asignado.

2. **Tipos de Permisos (Acciones):**

- **Lectura (r - Read):** Permite abrir y ver el contenido del archivo. En directorios, permite listar las carpetas y archivos internos.
- **Escritura (w - Write):** Permite modificar el contenido de un archivo o borrarlo. En directorios, habilita la creación, renombrado o eliminación de archivos en su interior.
- **Ejecución (x - Execute):** Permite correr el archivo como si fuese un programa ejecutable o un script de sistema. En un directorio, este permiso es crítico, ya que otorga el derecho de "entrar" a él (usar el comando cd).

Al visualizar los archivos con un listado detallado, estos permisos se expresan mediante una cadena de 10 caracteres (por ejemplo, drwxr-xr-x). El primer carácter define el tipo de recurso (- para archivos normales, d para directorios), y los 9 caracteres siguientes representan los tripletes rwx de permisos para el Dueño, Grupo y Otros de forma ordenada.

b.

i. **chmod (Change Mode)**

- **Funcionalidad:** Cambia o redefine los permisos de acceso de seguridad de un archivo o directorio.
- **Parámetros importantes:**

- **-R (recursivo):** Aplica el cambio de permisos de manera masiva a todos los archivos y subcarpetas contenidos dentro de un directorio.
- **Sintaxis simbólica:** Permite añadir (+), quitar (-) o asignar (=) permisos de forma directa (por ejemplo, `chmod u+x,g-w archivo.txt` añade ejecución al dueño y le quita escritura al grupo).

ii. **chown (Change Owner)**

- **Funcionalidad:** Cambia el usuario propietario (dueño) y/o el grupo de seguridad de un archivo o directorio.
- **Parámetros importantes:**
 - **-R (recursivo):** Cambia recursivamente la titularidad de los archivos y subcarpetas internas de un directorio.
 - **Sintaxis:** `chown [usuario]:[grupo] [archivo]` (por ejemplo, `chown root:root archivo` modifica ambos campos simultáneamente).

iii. **chgrp (Change Group)**

- **Funcionalidad:** Modifica únicamente el grupo asignado a un archivo o carpeta, sin alterar la cuenta del propietario.
- **Parámetros importantes:**
 - **-R (recursivo):** Cambia recursivamente la pertenencia de grupo en toda una jerarquía de carpetas.

c.

La notación octal es una forma matemática abreviada que utiliza el sistema binario para representar la combinación de permisos de un archivo en base 8 (del 0 al 7). Cada tipo de permiso tiene asignado un peso numérico binario inmutable:

- **Lectura (r):** Equivale a **4** (en binario 100).
- **Escritura (w):** Equivale a **2** (en binario 010).
- **Ejecución (x):** Equivale a **1** (en binario 001).

Para definir los permisos de cada una de las tres categorías de usuarios (Dueño, Grupo, Otros), se suman los valores individuales de los accesos permitidos. Esto da como resultado un número de tres dígitos:

- **7 (4+2+1):** Permisos totales (rwx).
- **6 (4+2):** Lectura y escritura (rw-).

- **5 (4+1):** Lectura y ejecución (r-x).
- **4 (4):** Únicamente lectura (r--).
- **0 (0):** Ningún permiso asignado (---).

Ejemplo: Ejecutar `chmod 750` archivo significa que el Dueño tendrá todos los permisos (7 = rwx), el Grupo tendrá lectura y ejecución (5 = r-x), y los Otros no tendrán ningún tipo de acceso (0 = ---).

d.

Sí, es posible. El usuario root (superusuario) posee el identificador UID 0. Al operar en el modo kernel o poseer privilegios de administración globales sobre el hardware y el espacio de direcciones, root elude por completo las restricciones del sistema de archivos y puede leer, escribir o ejecutar cualquier archivo, incluso si se le han quitado explícitamente todos los permisos.

e.

- **Full Path Name (Path Absoluto):** Es la ruta completa e inequívoca para localizar un archivo o directorio. Comienza estrictamente desde la raíz unificada del sistema (/). No importa en qué directorio esté parado el usuario, un path absoluto siempre apunta al mismo lugar físico.
 - **Ejemplo claro:** `/home/isocso/ISOCSO/isocso.txt` o `/etc/passwd`.
- **Relative Path Name (Path Relativo):** Es la ruta de localización que se calcula tomando como punto de partida la ubicación de trabajo actual del usuario. No comienza con /. Utiliza caracteres especiales como `.` (directorio actual) o `..` (subir un directorio padre).
 - **Ejemplo claro:** Si estás parado en `/home/isocso`, para entrar al archivo de la carpeta interna basta con escribir la ruta relativa `ISOCSO/isocso.txt` (o `./ISOCSO/isocso.txt`).

f.

- **Comando de ubicación actual:** Se utiliza el comando `pwd` (*Print Working Directory*).
- **Forma rápida de ingresar al directorio personal (home):** Sí, existe. GNU/Linux reserva el carácter especial `~` (virgulilla) para representar la ruta de tu home

personal. Al escribir simplemente `cd ~` (o solamente `cd` sin argumentos), el intérprete te traslada inmediatamente a tu directorio de usuario.

- **¿Se puede usar esta idea para otros directorios?:** Sí. El comodín `~` se puede usar como un "ancla" para estructurar de manera rápida rutas relativas a tu directorio personal sin necesidad de escribir la ruta absoluta completa `/home/nombre_usuario/`.
 - **Ejemplo:** Si estás trabajando en el directorio administrativo `/etc/apt/` y querés saltar directamente a tu carpeta de prácticas sin escribir la ruta completa, podés ejecutar: `cd ~/ISOC`

El Shell expandirá de forma transparente el comando a `cd /home/tu_usuario/ISOC`.

g.

Comando	Funcionalidad	Parámetros más importantes
umount	Unmount: Desmonta de forma lógica un dispositivo de almacenamiento o partición que se encuentra enlazada al árbol jerárquico.	-f: Fuerza el desmontaje de la unidad en caso de que un proceso esté utilizando activamente el FileSystem.
du	Disk Usage: Mide y reporta de forma recursiva el espacio real ocupado en disco por archivos y directorios.	-h: Muestra el espacio en formato legible (<i>human-readable</i>) en KB, MB o GB. -s: Muestra únicamente el total acumulado de la carpeta omitiendo la lista interna.
df	Disk Free: Muestra la cantidad total de espacio libre y ocupado en todas las particiones físicas montadas en el S.O..	-h: Muestra los bloques lógicos de disco en un formato legible en Gigabytes (GB). -T: Lista de manera explícita el tipo de sistema de archivos (ej. ext4, fat32).
mount	Mount: Enlaza físicamente una partición o un dispositivo físico a una carpeta específica (punto de montaje) del árbol de directorios.	-t [tipo]: Especifica el tipo de sistema de archivos del disco a montar (ej. ext4, ntfs). -o [opciones]: Define directivas de montaje adicionales, como <code>ro</code> (solo lectura) o <code>noexec</code> (impide ejecutar programas).
mkfs	Make File System: Formatea una partición del disco rígido construyendo sobre ella una estructura de sistema de archivos compatible.	-t [tipo]: Define el tipo de FileSystem a escribir (ej. <code>mkfs -t ext4 /dev/sdb1</code> inicializa la partición como Extended 4).
fdisk	Herramienta destructiva de particionamiento por consola de línea de comandos que permite crear, borrar, listar y	-l: Lista detalladamente las tablas de particiones y los dispositivos de almacenamiento detectados en la máquina.

	modificar tablas de particiones lógicas en discos rígidos.	/dev/device: Abre el menú interactivo para estructurar el disco.
write	Envía mensajes de texto de terminal en tiempo real de forma directa al monitor de otro usuario activo dentro de la misma máquina.	Sintaxis básica: write [nombre_usuario] [terminal_TTY].
losetup	Configura y asocia archivos de imágenes de disco regulares (como archivos .img o .iso) con dispositivos de bucle (<i>loop devices</i>), permitiendo tratarlos como si fuesen discos físicos normales.	- a : Muestra el listado de todos los loop devices que se encuentran asociados actualmente en el sistema. - d [loop] : Desvincula y libera el archivo del dispositivo de bucle.
stat	Despliega de forma detallada toda la información estructural, metadatos y estado físico de un archivo o directorio.	Muestra el número de inodo, permisos octales, UID, GID, tamaño y marcas temporales de acceso y modificación profunda.

16. Procesos:

a. ¿Qué significa que un proceso se está ejecutando en Background? ¿Y en Foreground?

b. ¿Cómo puedo hacer para ejecutar un proceso en Background? ¿Como puedo hacer para pasar un proceso de background a foreground y viceversa?

c. Pipe (|). ¿Cuál es su finalidad? Cite ejemplos de su utilización.

d. Redirección. ¿Qué tipo de redirecciones existen? ¿Cuál es su finalidad? Cite ejemplos de utilización.

a.

- **Foreground (Primer plano):** Es el modo de ejecución predeterminado de la Shell. Cuando lanzás un comando en foreground, este toma el control interactivo de tu terminal. No podés ingresar ningún otro comando ni interactuar con la línea de comandos hasta que el proceso actual finalice y te devuelva el prompt de la consola.
- **Background (Segundo plano):** Significa que el proceso se está ejecutando de forma asíncrona "detrás de escena". Al lanzarlo en background, la Shell te devuelve el prompt de comandos inmediatamente, permitiéndote seguir escribiendo y ejecutando otras tareas en la terminal en paralelo mientras el programa inicial continúa trabajando sin bloquear tu consola.

b.

- **Lanzar un proceso directamente en Background:** Para ejecutar un comando en segundo plano desde el principio, simplemente debés agregar el carácter especial **& (ampersand)** al final de la línea.

sleep 500 &

La terminal te responderá mostrando un número de trabajo entre corchetes (ej:) y el identificador de proceso (PID) asignado por el sistema.

- **Pasar un proceso de Background a Foreground:** Se utiliza el comando **fg** (*foreground*) seguido del identificador del trabajo (anteponiendo un %).

fg %1

- **Pasar un proceso de Foreground a Background (Viceversa):** Si lanzaste un proceso pesado en primer plano por error y tu terminal quedó bloqueada, realizás los siguientes pasos:

1. **Pausá el proceso:** Presioná la combinación de teclas Ctrl + Z en tu teclado. Esto envía una señal SIGSTOP al proceso, deteniéndolo temporalmente y liberando tu terminal. La consola te mostrará un mensaje indicando que el trabajo quedó *Detenido* o *Stopped*.
2. **Reanudalo en segundo plano:** Ejecutá el comando `bg` (*background*) seguido del número de trabajo.

`bg %1`

El proceso suspendido volverá a correr de manera activa, pero ahora lo hará en segundo plano, dejándote la terminal libre.

c.

- **Finalidad:** La tubería (Pipe o barra vertical |) tiene como finalidad conectar directamente la salida estándar (*stdout*) de un comando con la entrada estándar (*stdin*) de otro comando. Esto permite encadenar programas simples para realizar transformaciones de datos complejas en una sola línea, de manera eficiente y en tiempo real, evitando tener que generar y guardar archivos temporales en el disco duro.

- **Ejemplos de utilización:**

1. **Filtrar procesos activos en tu Debian:**

`ps aux | grep bash`

(El comando `ps` lista todos los procesos de la máquina y le pasa esa lista a `grep`, que filtra e imprime únicamente las líneas que contengan la palabra "bash").

- **Visualizar de forma paginada un listado largo:**

`ls -la /usr/bin | less`

17. Otros comandos de Linux (Indique funcionalidad y parámetros):

- a. ¿A qué hace referencia el concepto de empaquetar archivos en GNU/Linux?
- b. Seleccione 4 archivos dentro de algún directorio al que tenga permiso y sume el tamaño de cada uno de estos archivos. Cree un archivo empaquetado conteniendo estos 4 archivos y compare los tamaños de los mismos. ¿Qué característica nota?
- c. ¿Qué acciones debe llevar a cabo para comprimir 4 archivos en uno solo? Indique la secuencia de comandos ejecutados.
- d. ¿Pueden comprimirse un conjunto de archivos utilizando un único comando?
- e. Investigue la funcionalidad de los siguientes comandos:
 - i. tar
 - ii. grep
 - iii. gzip
 - iv. zgrep
 - v. wc

a.

En GNU/Linux, empaquetar significa agrupar varios archivos y/o carpetas dentro de un único archivo contenedor (denominado archivo *.tar* o *tarball*), preservando intacta la estructura jerárquica de los directorios, los nombres de archivos, sus permisos de seguridad, dueños, grupos y marcas temporales de modificación.

- **Diferencia clave con comprimir:** El empaquetado no realiza ninguna reducción en la redundancia de los datos (no achica el peso real de la información por sí mismo). Su único objetivo es facilitar la portabilidad, el transporte, el backup o la distribución de múltiples archivos unificándolos en uno solo.

b.

El archivo empaquetado resultante es notablemente más grande (casi 75 veces más pesado) que la suma de los tamaños de los archivos individuales.

Esto ocurre por dos motivos fundamentales del sistema operativo UNIX:

1. **Metadatos y Headers:** Por cada archivo que se introduce en el paquete, el comando tar añade un bloque de cabecera (header) fijo de 512 bytes que almacena los metadatos (permisos, propietario UID, grupo GID, inodo, tamaño,

etc.). Al meter 4 archivos, ya se pierden al menos 2.048 bytes puramente en cabeceras.

2. **Factor de Bloqueo (Blocking Factor):** Históricamente, tar (*Tape Archive*) se diseñó para escribir en cintas magnéticas de almacenamiento físico, las cuales leen y escriben en bloques óptimos de records. Por defecto, el factor de bloqueo estándar de GNU tar es de 20 bloques de 512 bytes, lo que da un registro mínimo de escritura de 10.240 bytes (10 KB). Si la suma total de datos + cabeceras no llega a esa medida, tar rellena el espacio restante con bytes nulos (\0), provocando que el archivo final tenga exactamente ese peso mínimo.

c.

Como las herramientas tradicionales de compresión en Linux (como gzip) solo comprimen archivos individuales de manera aislada (sin agruparlos), para comprimir 4 archivos en uno solo se deben combinar dos etapas secuenciales (empaquetar + comprimir):

1. **Paso 1: Empaquetar los archivos** en un único contenedor utilizando tar:

```
tar -cvf mis_archivos.tar file1.txt file2.txt file3.txt file4.txt
```

1. **Paso 2: Comprimir el paquete resultante** con la utilidad gzip:

```
gzip mis_archivos.tar
```

d.

Sí, absolutamente. Las implementaciones modernas de tar integran la capacidad de canalizar la salida del empaquetado a través de diferentes algoritmos de compresión en un solo paso mediante el uso de "flags" o parámetros dedicados:

- **Comando unificado utilizando Gzip (Algoritmo de compresión rápido):**

```
tar -zcvf comprimido.tar.gz file1.txt file2.txt file3.txt file4.txt
```

(Donde -z le ordena a tar que llame de forma transparente a gzip en memoria para comprimir el archivo resultante de manera inmediata).

e.

i. **tar (Tape Archive)**

- **Funcionalidad:** Herramienta utilizada para crear, listar, modificar y extraer archivos empaquetados.
- **Parámetros críticos:**
 - **-c (*create*):** Crea un nuevo archivo de empaquetado.
 - **-x (*extract*):** Extrae el contenido de un archivo empaquetado.
 - **-t (*list*):** Lista el contenido interno del paquete sin necesidad de extraerlo en el disco.
 - **-v (*verbose*):** Muestra detalladamente en pantalla el progreso y los nombres de los archivos procesados.
 - **-f (*file*):** Especifica el nombre del archivo de destino (debe ser siempre el último parámetro antes del nombre).
 - **-z (*gzip*):** Comprime o descomprime el archivo usando el algoritmo Gzip.

ii. grep (Global Regular Expression Print)

- **Funcionalidad:** Busca de manera exhaustiva patrones de texto o expresiones regulares dentro de uno o varios archivos de texto y muestra en pantalla únicamente las líneas que coincidan con la búsqueda.
- **Parámetros críticos:**
 - **-i:** Realiza una búsqueda insensible a mayúsculas y minúsculas (*case-insensitive*).
 - **-v:** Invierte el sentido de la búsqueda; muestra únicamente las líneas que no contienen el patrón especificado.
 - **-n:** Imprime el número de línea exacto dentro del archivo de texto donde se encontró la coincidencia.
 - **-r:** Realiza una búsqueda recursiva dentro de todos los subdirectorios y archivos de la ruta especificada.

iii. gzip (GNU zip)

- **Funcionalidad:** Reduce el tamaño de almacenamiento de archivos individuales utilizando el algoritmo de compresión de codificación Lempel-Ziv (LZ77). Reemplaza el archivo original por uno con extensión .gz.
- **Parámetros críticos:**
 - **-d (*decompress*):** Descomprime el archivo .gz devolviéndolo a su estado original.

- **-k (*keep*)**: Fuerza a mantener el archivo original intacto durante el proceso de compresión/descompresión (evita que se borre).
- **-9 (*best*)**: Indica al procesador que utilice el nivel máximo de compresión disponible (más lento pero genera un archivo final más pequeño).

iv. **zgrep**

- **Funcionalidad**: Es una utilidad que actúa como un "wrapper" de seguridad. Permite buscar patrones de texto (usando grep) directamente adentro de archivos que están comprimidos (.gz), realizando la descompresión al vuelo en la memoria RAM. Esto evita tener que descomprimir físicamente los archivos pesados en el disco para poder leerlos (esencial para analizar históricos de logs en /var/log).
- **Parámetros críticos**: Utiliza y acepta los mismos parámetros estándar que grep (como -i, -n, -v, etc.).

v. **wc (Word Count)**

- **Funcionalidad**: Cuenta y reporta la cantidad de líneas, palabras, caracteres y bytes contenidos en uno o más archivos de texto.
- **Parámetros críticos**:
 - **-l (*lines*)**: Cuenta de forma exclusiva la **cantidad de líneas** (muy útil al usar tuberías, ej: ls | wc -l cuenta cuántos archivos hay en una carpeta).
 - **-w (*words*)**: Cuenta el número de palabras.
 - **-c (*bytes*)**: Cuenta la cantidad total de bytes que pesa el texto.
 - **-m (*chars*)**: Cuenta la cantidad de caracteres (útil para codificaciones multibyte donde un caracter puede no equivaler a un byte).

18. Indique qué acción realiza cada uno de los comandos indicados a continuación considerando su orden. Suponga que se ejecutan desde un usuario que no es root ni pertenece al grupo de root. (Asuma que se encuentra posicionado en el directorio de trabajo del usuario con el que se logueó). En caso de no poder ejecutarse el comando, indique la razón:

`ls -l > prueba`

`ps > PRUEBA`

`chmod 710 prueba`

`chown root:root PRUEBA`

`chmod 777 PRUEBA`

`chmod 700 /etc/passwd`

`passwd root`

`rm PRUEBA`

`man /etc/shadow`

`find / -name * .conf`

`usermod root -d /home/ newroot -L`

`cd / root`

`rm *`

`cd / etc`

`cp * /home -R`

`shutdown`

1. `ls -l > prueba`

- **Acción:** Lista el contenido del directorio actual en formato largo (`ls -l`) y redirecciona la salida estándar (`>`) para guardarla dentro de un nuevo archivo de texto llamado `prueba`.
- **¿Se puede ejecutar?:** Sí. Al estar parado en tu propia carpeta `home`, tenés permisos totales de escritura para crear y modificar archivos dentro de ella.

2. `ps > PRUEBA`

- **Acción:** Muestra el listado de procesos de usuario activos en la terminal actual (ps) y redirecciona esa salida estándar para guardarla en un archivo llamado PRUEBA.
- **¿Se puede ejecutar?:** Sí. Se crea de forma exitosa en tu directorio personal.

3. chmod 710 prueba

- **Acción:** Modifica los permisos de acceso del archivo prueba. Otorga permisos totales al dueño (7 = rwx), permisos de ejecución al grupo (1 = --x), y quita todo acceso a otros (0 = ---).
- **¿Se puede ejecutar?:** Sí. Como vos creaste el archivo prueba en el primer paso, sos el propietario legítimo del archivo y el sistema te permite alterar sus bits de seguridad libremente.

4. chown root:root PRUEBA

- **Acción:** Intenta cambiar el propietario del archivo PRUEBA al usuario root y su grupo al grupo root.
- **¿Se puede ejecutar?:** No. En sistemas GNU/Linux, únicamente el superusuario root tiene el privilegio de ejecutar chown para cambiar el dueño de un archivo. Un usuario común no puede transferir la propiedad de sus recursos.
- **Razón del fallo:** La terminal devolverá el error Operation not permitted (Operación no permitida). El archivo PRUEBA conserva intacto su propietario original (tu usuario).

5. chmod 777 PRUEBA

- **Acción:** Intenta cambiar los permisos de PRUEBA para otorgar lectura, escritura y ejecución totales a todos los usuarios del sistema (rwxrwxrwx).
- **¿Se puede ejecutar?:** Sí. Como el comando anterior (chown) falló de forma segura, vos seguís siendo el dueño legítimo de PRUEBA, por lo que tu orden de cambiar los permisos se ejecuta con éxito.

6. chmod 700 /etc/passwd

- **Acción:** Intenta cambiar los permisos del archivo maestro de usuarios del sistema /etc/passwd para que solo root pueda interactuar con él.
- **¿Se puede ejecutar?:** No. El archivo /etc/passwd es un archivo crítico del sistema operativo y pertenece al usuario root. Un usuario normal no puede modificar sus metadatos de seguridad.
- **Razón del fallo:** Devolverá Permission denied (Permiso denegado).

7. passwd root

- **Acción:** Intenta cambiar o redefinir la contraseña de acceso de la cuenta del superusuario root.
- **¿Se puede ejecutar?:** No. Un usuario común solo tiene permitido ejecutar el comando passwd a secas para cambiar su propia contraseña. Alterar claves de terceros (especialmente de administración) es exclusivo del superusuario.
- **Razón del fallo:** Devolverá Permission denied o passwd: El usuario root no puede ser modificado.

8. rm PRUEBA

- **Acción:** Elimina físicamente el archivo PRUEBA del disco.
- **¿Se puede ejecutar?:** Sí. Como seguís siendo el propietario del archivo y está guardado en tu directorio home, podés removerlo sin inconvenientes.

9. man /etc/shadow

- **Acción:** Intenta abrir el manual de ayuda de la ruta /etc/shadow.
- **¿Se puede ejecutar?:** No de esa manera. El comando man está diseñado para recibir nombres de utilidades o configuraciones de la teoría (por ejemplo, man shadow para ver el formato), no rutas absolutas de archivos físicos.
- **Razón del fallo:** El comando fallará indicando No manual entry for /etc/shadow. *(Nota teórica: Si la intención del comando fuera leer el archivo real usando cat /etc/shadow, tampoco se podría por razones de seguridad, ya que /etc/shadow solo es legible por root).*

10. find / -name * .conf

- **Acción:** Busca de forma exhaustiva en todo el sistema de archivos (comenzando por la raíz /) cualquier archivo cuyo nombre termine con la extensión .conf.
- **¿Se puede ejecutar?:** Sí, pero de manera parcial. El comando se iniciará y listará las coincidencias encontradas únicamente en los directorios públicos o propios donde tu usuario tenga acceso.
- **Consecuencia de ejecución:** Por cada directorio protegido o administrativo del sistema al que no tengas derecho de entrar, la terminal arrojará una línea de error Permission denied (Permiso denegado), "ensuciando" la pantalla junto con los pocos resultados exitosos.

11. usermod root -d /home/ newroot -L

Acción: Intenta modificar la configuración de la cuenta de root para asignarle un nuevo directorio personal (-d /home/newroot) y bloquear su contraseña de acceso (-L).

- **¿Se puede ejecutar?:** No. El comando usermod es una herramienta exclusiva de administración del sistema.
- **Razón del fallo:** Devolverá Permission denied o usermod: Permission denied.

12. cd / root

- **Acción:** Intenta ingresar al directorio personal del administrador (/root).
- **¿Se puede ejecutar?:** No.
- **Razón del fallo:** La carpeta del superusuario tiene permisos sumamente restrictivos (drwx-----) y tu cuenta cae en la categoría de "Otros". El sistema de archivos deniega el acceso devolviendo Permission denied (Permiso denegado).

13. rm *

- **Acción:** Ordena borrar todos los archivos contenidos en el directorio actual.
- **¿Se puede ejecutar?:** Sí, pero con graves consecuencias personales. Como el comando anterior (cd /root) falló, tu terminal nunca cambió de ubicación y seguís parado en tu directorio personal (~). Al ejecutar rm *, eliminarás de forma definitiva todos los archivos de tu propia autoría que tenías guardados en tu home (incluyendo el archivo prueba creado en el paso 1).

14. cd / etc

- **Acción:** Modifica el directorio actual de trabajo para trasladarte a la carpeta /etc (donde residen los archivos de configuración del S.O.).
- **¿Se puede ejecutar?:** Sí. Aunque es una carpeta del sistema, /etc tiene permisos de lectura y ejecución públicos para que todas las aplicaciones del usuario puedan leer los archivos de configuración básica. *(Nota: Si se interpreta de forma literal con el espacio en blanco del error tipográfico de la guía cd / etc, el shell te llevará a la raíz / e ignorará la palabra etc).*

15. cp * /home -R

- **Acción:** Intenta copiar de forma recursiva (-R) todos los archivos contenidos en el directorio actual (que ahora es /etc) hacia el directorio de usuarios /home.
- **¿Se puede ejecutar?:** No. Aunque tenés permisos para leer los archivos de /etc, no poseés permisos de escritura sobre el directorio de destino /home, el cual es propiedad exclusiva de root.

- **Razón del fallo:** El comando fallará devolviendo ráfagas de errores de tipo Permission denied (Permiso denegado) al no poder crear los archivos de destino.

16. shutdown

- **Acción:** Ordena el apagado inmediato o programado del sistema operativo de la máquina.
- **¿Se puede ejecutar?:** No. Apagar o reiniciar el sistema de forma física o virtual es una tarea de administración crítica restringida para usuarios comunes en entornos de terminal multiusuario.
- **Razón del fallo:** La terminal devolverá un error indicando que debe ser superusuario para realizar esta acción o requerirá de autenticación interactiva de systemd que no poseés.

19. Indique qué comando sería necesario ejecutar para realizar cada una de las siguientes acciones:

a. Terminar el proceso con PID 23.

b. Terminar el proceso llamado init o systemd. ¿Qué resultados obtuvo?

c. Buscar todos los archivos de usuarios en los que su nombre contiene la cadena ".conf"

d. Guardar una lista de procesos en ejecución el archivo /home//procesos

e. Cambiar los permisos del archivo /home//xxxx a:

i. Usuario: Lectura, escritura, ejecución

ii. Grupo: Lectura, ejecución

iii. Otros: ejecución

f. Cambiar los permisos del archivo /home//yyyy a:

i. Usuario: Lectura, escritura.

ii. Grupo: Lectura, ejecución

iii. Otros: Ninguno

g. Borrar todos los archivos del directorio /tmp

h. Cambiar el propietario del archivo /opt/isodata al usuario isocso

i. Guardar en el archivo /home//donde el directorio donde me encuentro en este momento, en caso de que el archivo exista no se debe eliminar su contenido anterior.

a.

kill 23

b.

kill -9 1

- **Como usuario común:** Devuelve el error Permiso denegado (Permission denied).
- **Como root:** Se produce un colapso inmediato del sistema (Kernel Panic), congelando o reiniciando la máquina virtual de manera abrupta al finalizar el proceso principal del espacio de usuario (PID 1).

c.

```
find /home -name "*.conf"
```

d.

```
ps -ef > /home/<su_nombre_de_usuario>/procesos
```

e.

- **Usuario:** Lectura, escritura, ejecución (7)
- **Grupo:** Lectura, ejecución (5)
- **Otros:** ejecución (1)

f.

- **Usuario:** Lectura, escritura (6)
- **Grupo:** Lectura, ejecución (5)
- **Otros:** Ninguno (0)

g.

```
rm -rf /tmp/*
```

h.

```
chown isocso /opt/isodata
```

i.

```
pwd >> /home/<su_nombre_de_usuario>/donde
```

20. Indique qué comando sería necesario ejecutar para realizar cada una de las siguientes acciones:

- a. Ingrese al sistema como usuario "root"**
- b. Cree un usuario. Elija como nombre, por convención, la primera letra de su nombre seguida de su apellido. Asígnele una contraseña de acceso.**
- c. ¿Qué archivos fueron modificados luego de crear el usuario y qué directorios se crearon?**
- d. Crear un directorio en /tmp llamado miCursada**
- e. Copiar todos los archivos de /var/log al directorio antes creado.**
- f. Para el directorio antes creado (y los archivos y subdirectorios contenidos en él) cambiar el propietario y grupo al usuario creado y grupo users.**
- g. Agregue permiso total al dueño, de escritura al grupo y escritura y ejecución a todos los demás usuarios para todos los archivos dentro de un directorio en forma recursiva.**
- h. Acceda a otra terminal para loguearse con el usuario antes creado.**
- i. Una vez logueado con el usuario antes creado, averigüe cuál es el nombre de su terminal.**
- j. Verifique la cantidad de procesos activos que hay en el sistema.**
- k. Verifiqué la cantidad de usuarios conectados al sistema.**
- l. Vuelva a la terminal del usuario root y envíele un mensaje al usuario anteriormente creado enviándole que el sistema va a ser apagado.**
- m. Apague el sistema**

a.

su -

b.

adduser lcardo

c.

- **Archivos modificados:** /etc/passwd, /etc/shadow, /etc/group y /etc/gshadow.

- **Directorios creados:** El directorio personal del usuario en /home/lcardo (el cual almacena por defecto los archivos de su propiedad).

d.

```
mkdir /tmp/miCursada
```

e.

```
cp -R /var/log/* /tmp/miCursada/
```

f.

```
chown -R jgomez:users /tmp/miCursada
```

g.

- **Mediante notación octal:**

- *Dueño (rwx):* 7 (4 + 2 + 1)

- *Grupo (-w-):* 2 (0 + 2 + 0)

- *Otros (-wx):* 3 (0 + 2 + 1)

```
chmod -R 723 /tmp/miCursada
```

- **Mediante notación simbólica:**

```
chmod -R u=rwx,g=w,o=wx /tmp/miCursada
```

h.

- Presione la combinación de teclas Ctrl + Alt + F2 (o cualquier F3, F4, F5, F6) para saltar a una consola virtual de texto (TTY) independiente e inicie sesión ingresando las credenciales de lcardo.

i.

```
tty
```

j.

`ps -ef | wc -l`

k.

`who | wc -l`

l.

1. Verifique el nombre de la terminal (TTY) donde está conectado el usuario (por ejemplo, tty2) ejecutando `who`.
2. Ejecute el comando de mensajería:

`write lcardo tty2`

1. Escriba el mensaje en la consola: "El sistema va a ser apagado."
2. Presione la combinación de teclas `Ctrl + D` para finalizar la escritura y enviar el mensaje de texto.

m.

`shutdown -h now`

21. Indique qué comando sería necesario ejecutar para realizar cada una de las siguientes acciones:

a. Cree un directorio cuyo nombre sea su número de legajo e ingrese a él.

b. Cree un archivo utilizando el editor de textos vi, e introduzca su información personal: Nombre, Apellido, Número de alumno y dirección de correo electrónico. El archivo debe llamarse "LEAME".

c. Cambie los permisos del archivo LEAME, de manera que se puedan ver reflejados los siguientes permisos:

➤ **Dueño: ningún permiso**

➤ **Grupo: permiso de ejecución**

➤ **Otros: todos los permisos**

d. Vaya al directorio /etc y verifique su contenido. Cree un archivo dentro de su directorio personal cuyo nombre sea leame donde el contenido del mismo sea el listado de todos los archivos y directorios contenidos en /etc. ¿Cuál es la razón por la cuál puede crear este archivo si ya existe un archivo llamado "LEAME" en este directorio?

e. ¿Qué comando utilizaría y de qué manera si tuviera que localizar un archivo dentro del filesystem? ¿Y si tuviera que localizar varios archivos con características similares? Explique el concepto teórico y ejemplifique.

f. Utilizando los conceptos aprendidos en el punto anterior, busque todos los archivos cuya extensión sea .so y almacene el resultado de esta búsqueda en un archivo dentro del directorio creado en el primer inciso. El archivo deberá llamarse ejercicioF

a.

`mkdir <legajo> && cd <25563/8>`

b.

`vi LEAME`

c.

`chmod 017 LEAME`

d.

```
cd /etc
```

```
ls -la
```

```
ls -la /etc > ~/leame
```

Razón por la cual pueden coexistir "LEAME" y "leame": GNU/Linux es un sistema operativo sensible a mayúsculas y minúsculas (*case-sensitive*). Para el sistema de archivos (FileSystem), los nombres de archivo "LEAME" (en mayúsculas) y "leame" (en minúsculas) representan dos identificadores de inodo y entidades de almacenamiento completamente distintas. Por lo tanto, pueden coexistir de manera simultánea en el mismo directorio de trabajo sin producir ningún conflicto de duplicidad ni sobrescritura.

e.

En GNU/Linux existen dos comandos principales para localizar archivos en el disco, cada uno bajo un principio teórico de funcionamiento distinto:

1. Comando find (Búsqueda en tiempo real)

- **Concepto teórico:** Realiza una búsqueda física y recursiva recorriendo de manera directa el árbol de directorios del sistema de archivos en tiempo real. Es un método sumamente preciso y dinámico (encuentra archivos creados hace instantes), pero puede consumir más recursos de lectura de disco en búsquedas globales extensas.
- **Ejemplo para localizar un archivo específico:** `find /home -name "LEAME"`
- **Ejemplo para buscar varios archivos similares (usando patrones de coincidencia):** `find /etc -name "*.conf"`

2. Comando locate (Búsqueda por base de datos indexada)

- **Concepto teórico:** Busca las rutas de forma instantánea dentro de una base de datos local previamente construida e indexada por el sistema operativo (actualizada comúnmente con el comando `updatedb`). Su ejecución es extremadamente veloz porque no realiza accesos directos al disco físico, pero posee la desventaja de que no localizará archivos creados de manera muy reciente si la base de datos no se ha actualizado.
- **Ejemplo para localizar un archivo específico o patrón:** `locate LEAME`

22. Indique qué acción realiza cada uno de los comandos indicados a continuación considerando su orden. Suponga que se ejecutan desde un usuario que no es root ni pertenece al grupo de root. (Asuma que se encuentra posicionado en el directorio de trabajo del usuario con el que se logueó). En caso de no poder ejecutarse el comando indique la razón:

01. mkdir iso
02. cd . / iso; ps > f0
03. ls > f1
04. cd /
05. echo \$HOME
06. ls -l \$> \$HOME/ iso/ls
07. cd \$HOME; mkdir f2
08. ls -ld f2
09. chmod 341 f2
10. touch dir
11. cd f2
12. cd ~/iso
13. pwd > f3
14. ps | grep 'ps' | wc -l >> ../f2/f3
15. chmod 700 ../f2 ; cd ..
16. find . -name etc/passwd
17. find / -name etc/passwd
18. mkdir ejercicio5
19.
20.

a. Inicie 2 sesiones utilizando su nombre de usuario y contraseña. En una sesión vaya siguiendo paso a paso las órdenes que se encuentran escritas en el cuadro superior.

En la otra sesión, cree utilizando algún editor de textos un archivo que se llame

"explicacion_de_ejercicio" dentro del directorio creado en el ejercicio 22 y, para cada una de los comandos que ejecute en la otra sesión, realice una breve explicación de los resultados obtenidos.

b. Complete los comandos 19 y 20, de manera tal que realicen la siguiente acción:

19: Copiar el directorio iso y todo su contenido al directorio creado en 24.a

20: Copiar el resto de los archivos y directorios que se crearon en este ejercicio al directorio creado en el ejercicio 24.a

c. Ejecute las órdenes 19 y 20 y coméntelas en el archivo creado en el inciso a).

01. mkdir iso

- **Acción:** Crea un directorio llamado iso en el directorio actual.
- **¿Se ejecuta?:** Sí. El usuario tiene permisos de escritura en su propio directorio personal.

02. cd . /iso; ps > f0

- **Acción:** Cambia con éxito al directorio iso y guarda la lista de procesos en el archivo f0 dentro de esa carpeta.
- **¿Se ejecuta?:** Sí.

03. ls > f1

- **Acción:** Guarda el listado del contenido del directorio actual en el archivo f1.
- **¿Se ejecuta?:** Sí. Se crea dentro de la carpeta iso (si se aplicó la corrección del paso anterior) o en el directorio personal.

04. cd /

- **Acción:** Cambia el directorio de trabajo actual a la raíz del sistema de archivos (/).
- **¿Se ejecuta?:** Sí. Todos los usuarios tienen permisos de lectura y ejecución en la raíz.

05. echo \$HOME

- **Acción:** Muestra en pantalla el valor de la variable de entorno que define la ruta del directorio personal del usuario (ej. /home/usuario).
- **¿Se ejecuta?:** Sí.

06. ls -l \$> \$HOME/ iso/ls

- **Falla por error de sintaxis en el intérprete (bash: error de sintaxis cerca del elemento inesperado '\$'), debido al carácter de redirección incorrecto \$>.**

07. cd \$HOME; mkdir f2

- **Acción:** Regresa al directorio personal del usuario y crea un nuevo directorio llamado f2.
- **¿Se ejecuta?:** Sí.

08. ls -ld f2

- **Acción:** Muestra en formato largo los metadatos y permisos del directorio f2 en sí mismo, sin listar lo que tiene adentro.
- **¿Se ejecuta?:** Sí.

09. chmod 341 f2

- **Acción:** Cambia los permisos de la carpeta f2 a 341 en octal (Dueño: -wx, Grupo: r--, Otros: --x).
- **¿Se ejecuta?:** Sí. Como eres el propietario de f2, tienes derecho a modificar sus bits de seguridad.

10. touch dir

- **Acción:** Crea un archivo vacío llamado dir en el directorio personal del usuario.
- **¿Se ejecuta?:** Sí.

11. cd f2

- **Acción:** Intenta ingresar al directorio f2.
- **¿Se ejecuta?:** Sí. Para entrar a un directorio se necesita el permiso de ejecución (x), el cual está activo para el dueño (permiso 3 = -wx). *Nota: no podrás listar su contenido con ls al no tener permiso de lectura (r), pero sí posicionarte en él.*

12. cd ~/iso

- **Acción:** Cambia de directorio a la carpeta iso ubicada en tu directorio personal.
- **¿Se ejecuta?:** Sí.

13. pwd > f3

- **Acción:** Ejecuta el comando pwd (ruta actual) y guarda el resultado en el archivo f3 dentro de iso.
- **¿Se ejecuta?:** Sí.

14. `ps | grep 'ps' | wc -l >> ../f2/f3`

- **Acción:** Filtra y cuenta los procesos activos que contienen la palabra 'ps' y anexa el número resultante al final de un archivo llamado f3 dentro de la carpeta f2.
- **¿Se ejecuta?:** Sí. Se ejecuta correctamente debido a que el dueño tiene permisos de escritura y ejecución (3 = -wx) sobre la carpeta destino f2, lo que le permite crear y escribir nuevos archivos en su interior.

15. `chmod 700 ../f2 ; cd ..`

- **Acción:** Restablece los permisos de la carpeta f2 a 700 (permisos totales rwx exclusivos para el dueño) y luego retrocede un nivel para volver a posicionarse en el directorio personal.
- **¿Se ejecuta?:** Sí.

16. `find . -name etc/passwd`

- **Acción:** Busca de forma recursiva a partir del directorio actual (el home) un archivo llamado exactamente etc/passwd.
- **¿Se ejecuta?:** Sí, pero no arrojará ningún resultado, ya que ese archivo reside en la raíz / del sistema operativo y no dentro de tu carpeta personal.

17. `find / -name etc/passwd`

- **Acción:** Busca desde la raíz / de forma recursiva un archivo o directorio llamado etc/passwd.
- **¿Se ejecuta?:** Sí, de forma parcial. Encontrará el archivo, pero arrojará ráfagas de errores Permission denied (Permiso denegado) en pantalla cada vez que intente leer directorios protegidos del sistema a los que tu usuario común no tiene acceso.

18. `mkdir ejercicio5`

- **Acción:** Crea un directorio llamado ejercicio5 en tu directorio personal.
- **¿Se ejecuta?:** Sí.

b.

19. `cp -R $HOME/iso /tmp/logs/`

20. `cp -R $HOME/f2 $HOME/dir $HOME/ejercicio5 /tmp/logs/`

23. Cree una estructura desde el directorio /home que incluya varios directorios, subdirectorios y archivos, según el esquema siguiente.



Asuma que “usuario” indica cuál es su nombre de usuario. Además deberá tener en cuenta que dirX hace referencia a directorios y fX hace referencia a archivos.

Utilizando la estructura de directorios anteriormente creada, indique qué comandos son necesarios para realizar las siguientes acciones:

- Mueva el archivo "f3" al directorio de trabajo /home/usuario.
- Copie el archivo "f4" en el directorio "dir11".
- Haga lo mismo que en el inciso anterior pero el archivo de destino, se debe llamar "f7".
- Cree el directorio copia dentro del directorio usuario y copie en él, el contenido de "dir1".
- Renombre el archivo "f1" por el nombre archivo y vea los permisos del mismo.
- Cambie los permisos del archivo llamado archivo de manera de reflejar lo siguiente:
 - Usuario: Permisos de lectura y escritura
 - Grupo: Permisos de ejecución
 - Otros: Todos los permisos
- Renombre los archivos "f3" y "f4" de manera que se llamen "f3.exe" y "f4.exe" respectivamente.
- Utilizando un único comando cambie los permisos de los dos archivos renombrados en el inciso anterior, de manera de reflejar lo siguiente:
 - Usuario: Ningún permiso
 - Grupo: Permisos de escritura
 - Otros: Permisos de escritura y ejecución

24. Indique qué comando/s es necesario para realizar cada una de las acciones de la siguiente secuencia de pasos (considerando su orden de aparición):

- a. Cree un directorio llamado logs en el directorio /tmp.**
- b. Copie todo el contenido del directorio /var/log en el directorio creado en el punto anterior.**
- c. Empaquete el directorio creado en a), el archivo resultante se debe llamar "misLogs.tar".**
- d. Empaquete y comprima el directorio creado en a), el archivo resultante se debe llamar "misLogs.tar.gz".**
- e. Copie los archivos creados en c) y d) al directorio de trabajo de su usuario.**
- f. Elimine el directorio creado en a), logs.**
- g. Desempaquete los archivos creados en c y d en 2 directorios diferentes**

a.

```
mkdir /tmp/logs
```

b.

```
cp -R /var/log/* /tmp/logs/
```

c.

```
tar -cf misLogs.tar /tmp/logs
```

d.

```
tar -zcf misLogs.tar.gz /tmp/logs
```

e.

```
cp misLogs.tar misLogs.tar.gz ~
```

f.

```
rm -rf /tmp/logs
```

g.

```
mkdir ~/ext_tar ~/desc_gz
```

```
tar -xf ~/misLogs.tar -C ~/ext_tar
```

```
tar -zxf ~/misLogs.tar.gz -C ~/desc_gz
```